TRMS Kontrol: Sıfırdan Uzmanlığa Teknik Rehber

Oct 21, 2026 · @Aybars

Genel bakış

60 saniyelik simüle edilmiş bir testte Doğrusal Olmayan Model Öngörülü Kontrol (N-MPC), otomatik ayarlanmış bir PID'ye göre integral izleme hatasını pitch ekseninde yaklaşık 5,4 kat, yaw ekseninde 4,4 kat azalttı. N-MPC altı referans basamağının hepsinde oturdu; PID yalnızca birinde oturdu.

Bu rehber, bu sonucu temel ilkelerden yola çıkarak açıklar. Okuyucunun kalkülüs, lineer cebir ve temel Newton mekaniği bildiğini, ancak kontrol teorisi bilmediğini varsayar. Sonunda sistem denklemlerini türetebilmeli, iki kontrol yasasını da yazabilmeli, kodun her satırını takip edebilmeli ve programın ürettiği her sayıyı ve grafiği okuyabilmelisiniz.

Çalışma ne yapıyor

Proje, Feedback Instruments 33-220 modelinin 6 durumlu doğrusal olmayan modelini kullanarak bir laboratuvar helikopter düzeneği olan İkiz Rotorlu MIMO Sistemi'ni (TRMS) simüle eder. İki kontrolcü, kirişin pitch ve yaw açılarındaki aynı basamak değişimleri dizisini izlemesini sağlamaya çalışır:

1. merkezi olmayan (decentralised) bir **PID**: bir PID döngüsü pitch'i ana rotor üzerinden, ikincisi yaw'ı kuyruk rotoru üzerinden sürer;
2. bir **N-MPC**: her örneklemede, tam doğrusal olmayan modeli kullanarak sonraki 1,9 saniye için bir optimizasyon problemi çözer.

Her iki kontrolcü de aynı optimizasyon algoritması tarafından aynı maliyet fonksiyonuna göre otomatik olarak ayarlanır, ardından ayrı bir 60 saniyelik değerlendirme senaryosunda çalıştırılır. Program her durumu ve her kontrolcü değişkenini kaydeder ve standart performans metriklerini hesaplar.

Depoya son eklenenler

Bu Türkçe sürüm, GitHub deposunun 14 Ekim 2026 tarihli son commit'indeki (6dc32ab) halini anlatır. İngilizce rehbere göre yeni olanlar:

- **Tek kontrolcü modu.** `python main.py --controllers nmpc` (veya `pid`) yalnızca seçilen kontrolcüyü ayarlar ve çalıştırır; karşılaştırma dosyaları yazılmaz, ama çalıştırmanın `summary.md` dosyası yine oluşur.
- **N-MPC çalışmaları.** `studies/` paketi: tahmin ufku taraması (`horizon_sweep`) ve model uyumsuzluğuna dayanıklılık (`robustness`).
- **3B web simülasyonu.** `python -m studies.animate_3d --open`, bir çalıştırmayı dönen rotorlarla 3B olarak tekrar oynatan, çevrimdışı çalışan tek dosyalık bir web sayfası üretir.
- **Windows düzeltmesi.** Tüm metin dosyaları artık açıkça UTF-8 olarak okunup yazılıyor; Türkçe Windows'ta yaşanan kodlama hatası böylece giderildi.

Bu rehber nasıl düzenlendi

Bölüm	Yanıtladığı soru
1. Teorik temeller	Fizik nedir; PID, N-MPC ve otomatik ayarlama matematiksel olarak nasıl çalışır?
2. Kod mimarisi	Hangi dosya ne yapar, veri nasıl akar, CasADi, IPOPT ve SciPy nasıl kullanılır, önbellek nasıl çalışır?
3. Veri sözlüğü ve metrikler	Her çıktı dosyası neyi içerir; her metriğin kesin tanımı nedir?
4. Görsel analiz	Üretilen her grafik nasıl okunur?
5. N-MPC çalışmaları ve 3B web simülasyonu	Ufuk ve model hatası N-MPC'yi nasıl etkiler; 3B tekrar oynatma nasıl kullanılır?
6. Sonuçlar, sınırlamalar, sözlük	Sayılar ne anlama gelir, neyi kanıtlamaz?

Başlamadan önce üç kapsam notu

Bu notlar, bu tür projelerle ilgili yaygın varsayımları düzeltir; rehberin geri kalanı bunlarla tutarlıdır.

- **Kütüphaneler.** N-MPC doğrudan CasADi ile kurulur ve IPOPT ile çözülür. do-mpc kütüphanesi kullanılmaz. Otomatik ayarlama optimizasyonunu SciPy sağlar.
- **Kısıtlar.** N-MPC yalnızca iki giriş üzerinde kesin sınırlar uygular. Durum kısıtları teoride açıklanır, ancak bu kodda hiçbiri uygulanmaz.

- **Öngörü.** N-MPC, kirişin kendi planladığı girişlere nasıl tepki vereceğini öngörür. Gelecekteki referans basamakları kendisine bildirilmez (`reference_preview: false`); bu yüzden bir basamağa, tıpkı PID gibi, ancak basamak gerçekleştiğinde tepki verir.

Rehberde “pitch” (kirişin yukarı-aşağı eğimi, yunuslama) ve “yaw” (sağa-sola dönüş, sapma) terimleri, kodda ve grafiklerde de böyle geçtiği için İngilizce bırakıldı. Programın ürettiği grafiklerin etiketleri İngilizcedir; metin her grafiği Türkçe açıklar.

Bölüm 1.1 — TRMS: fizik ve doğrusal olmayan model

TRMS, 2 girişli ve 2 çıkışlı bir sistemdir; altı iç durumu trigonometrik, polinom ve jiroskopik terimlerle birbirine bağlıdır. Bu terimler sistemi doğrusal olmayan ve çapraz bağlanımlı yapar: her giriş iki çıkışı da hareket ettirir.

İkiz Rotorlu MIMO Sistemi nedir

TRMS, sabit bir taban üzerinde orta noktasından mafsallı bir kiriştir. Mafsal, kirişin iki eksen etrafında dönmesine izin verir:

- **pitch**, alpha_v: düşey düzlemdeki dönüş (burun yukarı veya aşağı);
- **yaw**, alpha_h: yatay düzlemdeki dönüş (sola veya sağa dönüş).

Kirişin bir ucundaki **ana rotor** çoğunlukla düşey yönde iter ve pitch'i kontrol eder. Diğer ucundaki **kuyruk rotoru** çoğunlukla yatay yönde iter ve yaw'ı kontrol eder. Kısa bir kol üzerindeki karşı ağırlık, kirişin dinlenme duruşunu belirler. Her rotor, gerilimi kontrol girişi olan bir DC motorla sürülür. Girişler −1 ile +1 aralığına normalleştirilmiştir ve u_v (ana) ile u_h (kuyruk) olarak adlandırılır.

“MIMO”, *çok girişli çok çıkışlı* (multiple-input multiple-output) anlamına gelir. Burada 2 giriş ve 2 ölçülen çıkış vardır ve hiçbir giriş yalnızca tek bir çıkışa etki etmez. Laboratuvarlar TRMS'yi, gerçek bir helikopterin sorunlarını masa üstü ölçeğinde ürettiği için kullanır: kararlı görünmeyen az sönümlü hareket, eyleyici gecikmesi, doygunluk ve güçlü eksen bağlanımı.

Durum vektörü

Model altı durum ve iki giriş kullanır.

Sembol	Koddaki adı	Anlamı	Birim
x1 = alpha_v	`pitch`	pitch açısı	rad

Sembol	Koddaki adı	Anlamı	Birim
x2 = S_v	`S_v`	pitch momentum durumu (aşağıda tanımlanır)	rad/s
x3 = u_vv	`u_vv`	ana motorun iç (gecikmeli) gerilimi	–
x4 = alpha_h	`yaw`	yaw açısı	rad
x5 = S_h	`S_h`	yaw momentum durumu	rad/s
x6 = u_hh	`u_hh`	kuyruk motorunun iç (gecikmeli) gerilimi	–
u1 = u_v	`u_pitch`	ana motor komutu	–
u2 = u_h	`u_yaw`	kuyruk motoru komutu	–

Doğrusal olmayan durum denklemleri

Tam model, dx/dt = f(x, u) biçiminde şöyledir:

$$
\begin{aligned}
\dot{\alpha}_v &= \Omega_v = S_v + \frac{J_{tr}\,\omega_t(u_{hh})}{J_v} \\
\dot{S}_v &= \frac{1}{J_v}\Big[\, l_m S_f F_v\big(\omega_m(u_{vv})\big) + g\big((A-B)\cos\alpha_v - C\sin\alpha_v\big) - \Omega_h^2 H \sin\alpha_v \cos\alpha_v - K_v\,\Omega_v \Big] \\
\dot{u}_{vv} &= \frac{-u_{vv} + K_{mr}\,u_v}{T_{mr}} \\
\dot{\alpha}_h &= \Omega_h = S_h + \frac{J_{mr}\,\omega_m(u_{vv})\cos\alpha_v}{J_h(\alpha_v)} \\
\dot{S}_h &= \frac{1}{J_h(\alpha_v)}\Big[\, l_t S_f F_h\big(\omega_t(u_{hh})\big)\cos\alpha_v - K_h\,\Omega_h \Big] \\
\dot{u}_{hh} &= \frac{-u_{hh} + K_{tr}\,u_h}{T_{tr}} \\
J_h(\alpha_v) &= D\cos^2\alpha_v + E\sin^2\alpha_v + F
\end{aligned}
$$

Omega_v ve Omega_h kirişin gerçek açısal hızlarıdır. Kod bunları `pitch_rate` ve `yaw_rate` olarak kaydeder.

Denklemleri terim terim okumak

Dönme için Newton'un ikinci yasası. Her momentum denklemi "eyümsizlik × açısal ivme = torkların toplamı" biçimindedir. Pitch tork toplamının dört bileşeni vardır ve kod her birini ayrı kaydeder:

$$J_v\,\dot{S}_v = \underbrace{l_m S_f F_v}_{M_{aero,v}} + \underbrace{g\big((A-B)\cos\alpha_v - C\sin\alpha_v\big)}_{M_{grav}} \underbrace{-\,\Omega_h^2 H \sin\alpha_v\cos\alpha_v}_{M_{cent}} \underbrace{-\,K_v\Omega_v}_{M_{fric,v}}$$

- **Aerodinamik tork, M_aero,v** (`torque_aero_pitch`). Ana rotorun S_f·F_v itkisi (newton cinsinden) l_m = 0,24 m'lik moment kolunda etki eder.
- **Yerçekimi torku, M_grav** (`torque_gravity_pitch`). Kiriş dengeli değildir. A, B ve C sırasıyla kuyruk tarafının, ana rotor tarafının ve karşı ağırlık ile kirişin topaklanmış kütle momentleridir. Kollar kirişle birlikte döndüğü için düşey ağırlıklar cos alpha_v ve sin alpha_v ile orantılı torklar üretir. Bu, yerçekimi terimini kirişi dinlenme açısına çeken doğrusal olmayan bir yay gibi davrandırır.
- **Merkezkaç torku, M_cent** (`torque_centrifugal_pitch`). Kiriş Omega_h hızıyla yaw ekseninde döndüğünde kütleleri dışa doğru savrulur. Bu dışa çekme, Omega_h² ve sin(2 alpha_v) ile orantılı bir pitch torku üretir.
- **Viskoz sürtünme, M_fric,v** (`torque_friction_pitch`). Yatak ve hava sürüklemesi, harekete açısal hızla orantılı olarak, K_v katsayısıyla karşı koyar.

Yaw denklemi aynı yapıdadır ve iki tork içerir; bunlar `torque_aero_yaw` ve `torque_friction_yaw` olarak kaydedilir. Kuyruk itkisi l_t = 0,25 m'lik kolda etki eder. cos alpha_v ile çarpılır, çünkü eğik bir kirişte kuyruk itkisinin yalnızca yatay bileşeni kirişi yaw ekseninde döndürür. Yaw eyümsizliği J_h pitch'e bağlıdır, çünkü eğik bir kirişin kütleleri düşey eksene daha yakındır.

Eyleyiciler: motor gecikmesi ve rotor karakteristikleri

Hiçbir DC motor hızını anında değiştiremez. Model bunu birinci dereceden bir gecikme olarak temsil eder. İç durum u_vv (veya u_hh), komutu ana motor için T_mr = 1,432 s, kuyruk motoru için T_tr = 0,3842 s zaman sabitiyle izler. Bir zaman sabiti sonra iç durum, komuttaki bir basamağın %63'ünü kat etmiş olur. Dolayısıyla ana motor kuyruk motorundan yaklaşık dört kat yavaştır; bu, pitch kontrolü için çok önemlidir.

Gecikmeli gerilim, ölçülmüş bir **statik karakteristik** üzerinden rotor hızını, hız da ikinci bir karakteristik üzerinden itkiyi belirler. İkisi de laboratuvar verisine uydurulmuş polinomlardır; katsayılar en yüksek kuvvetten başlayarak sıralanır:

$$\begin{aligned}
\omega_m(u) &= 90.90u^6 + 599.73u^5 - 129.26u^4 - 1238.64u^3 + 63.45u^2 + 1238.41u \quad [\mathrm{rad/s}] \\
F_v(\omega) &= -3.48\times10^{-12}\omega^5 + 1.09\times10^{-9}\omega^4 + 4.123\times10^{-6}\omega^3 - 1.632\times10^{-4}\omega^2 + 9.544\times10^{-2}\omega \\
\omega_t(u) &= 2020u^5 + 194.69u^4 - 4283.15u^3 - 262.87u^2 + 3796.83u \quad [\mathrm{rad/s}] \\
F_h(\omega) &= -3\times10^{-14}\omega^5 + 1.595\times10^{-11}\omega^4 + 2.511\times10^{-7}\omega^3 - 1.808\times10^{-4}\omega^2 + 0.0801\,\omega
\end{aligned}$$

S_f ölçek çarpanı F'yi newton'a çevirir. Ana giriş tam açıkken (u = 1) ana rotor yaklaşık 625 rad/s (yaklaşık 6.000 rpm) hızla döner ve yaklaşık 0,70 N itki üretir. Girişten itkiye ilişki güçlü biçimde eğriseldir; u'daki aynı değişim farklı çalışma noktalarında farklı itki değişimi üretir. Sistemin doğrusal olmamasının başlıca kaynağı budur.

Çapraz bağlanım: her giriş neden iki ekseni de hareket ettirir

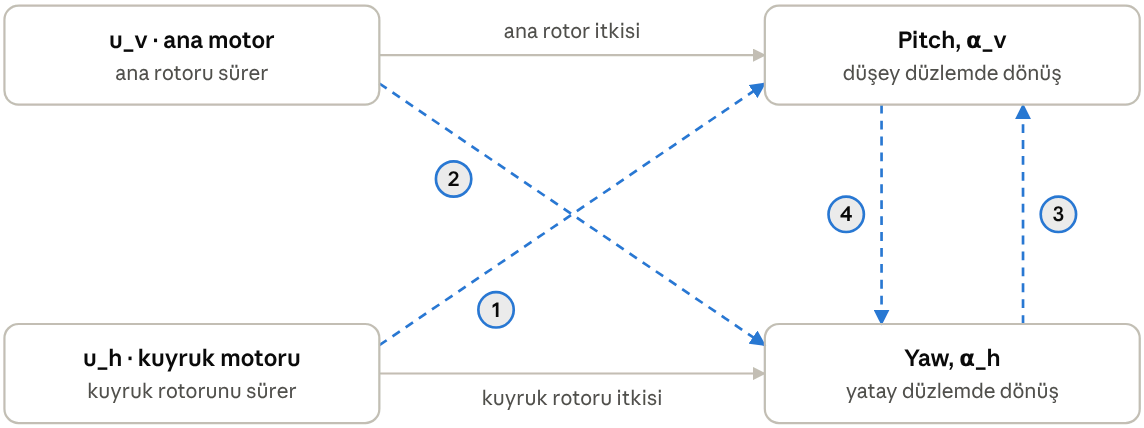
Çapraz bağlanım, bir girişin iki çıkışı da etkilemesidir. Modelde aşağıdaki diyagramda çizilen dört bağlanım yolu vardır.

1. **Kuyruk rotoru → pitch (tepki momentumu).** J_tr·omega_t/J_v terimi doğrudan pitch hızında yer alır. S_v kirişin açısal momentumunu tutar ve sıçrama yapamaz. Bu yüzden kuyruk rotoru hızlandığında, açısal momentumun korunumu kirişin pitch hızını anında değiştirir. Bu, Newton'un üçüncü yasasıdır: bir rotoru bir yöne döndürmek gövdeyi öbür yöne iter. Sonuçlarda her yaw basamağı, N-MPC'nin pitch'ini 17°'ye kadar sarsar.
2. **Ana rotor → yaw (tepki momentumu).** Benzer şekilde J_mr·omega_m·cos(alpha_v)/J_h yaw hızına girer. Ana rotor hızını değiştirmek kirişi yaw ekseninde döndürür.
3. **Yaw → pitch (merkezkaç).** Yaw dönüşü −Omega_h²·H·sin(alpha_v)·cos(alpha_v) pitch torkunu oluşturur.
4. **Pitch → yaw (geometri).** Pitch açısı, yararlı kuyruk itkisini cos alpha_v ile ölçekler ve yaw eyümsizliğini J_h(alpha_v) değiştirir.

Pitch ve yaw'ı iki bağımsız problem olarak ele alan bir kontrolcü bu dört yolun hepsini yok sayar. Bu nedenle etkilerini ancak gerçekleştikten sonra, açıklanamayan bozucular olarak görür.

Her giriş iki ekseni de hareket ettirir

Düz çizgiler: istenen etki · kesikli çizgiler: dört çapraz bağlanım yolu



1 · Kuyruk rotoru → pitch: tepki momentumu ($J_{tr}\omega_t / J_v$), anında etki eder

2 · Ana rotor → yaw: tepki momentumu ($J_{mr}\omega_m \cos\alpha_v / J_h$)

3 · Yaw → pitch: merkezkaç torku, Ω_h^2 ile orantılı

4 · Pitch → yaw: kuyruk itkisi $\cos\alpha_v$ ile ölçeklenir, $J_h(\alpha_v)$ değişir

çapraz bağlanım · iki giriş, iki eksen, dört yol

Fiziksel parametreler

Tüm değerler arXiv:1306.4883 makalesinin Tablo I'inden alınmıştır. Tek değişiklik F_h'nin doğrusal katsayısıdır: makalede basılan 0,8080 değeri kuyruk rotorunu ana rotordan yaklaşık dört kat güçlü yapardı; bu yüzden kod 0,0801 kullanır. Bölüm 5'teki dayanıklılık çalışması bu düzeltmeyi destekler: basılmış değerle döngü sürekli salınıma girer.

Sembol	Değer	Birim	Anlamı
A	0,0946875	kg·m	kuyruk tarafı yerçekimi moment terimi
B	0,11046	kg·m	ana rotor tarafı yerçekimi moment terimi
C	0,01986	kg·m	karşı ağırlık ve kiriş yerçekimi terimi
D, E, F	0,04988; 0,004745; 0,006230	kg·m²	yaw eyümsizlik katsayıları
H	0,048210	kg·m²	merkezkaç katsayısı

Sembol	Değer	Birim	Anlamı
J_v	0,055448	kg·m²	pitch eyümsizliği
J_mr	1,6543 × 10⁻⁵	kg·m²	ana rotor ve motor eyümsizliği
J_tr	2,65 × 10⁻⁵	kg·m²	kuyruk rotoru ve motor eyümsizliği
l_m, l_t	0,24; 0,25	m	ana ve kuyruk rotor kolları
S_f	8,43318 × 10⁻⁴	–	itki ölçek çarpanı
T_mr, T_tr	1,432; 0,3842	s	motor zaman sabitleri
K_mr, K_tr	1; 1	–	motor statik kazançları
K_v, K_h	0,0095; 0,00545371	N·m·s/rad	viskoz sürtünme
g	9,81	m/s²	yerçekimi ivmesi

(Makale A–C değerlerini kg·m² olarak verir; g ile çarpıldıklarında N·m cinsinden tork verirler ve kod onları bu şekilde kullanır.)

Denge, trim ve erişilebilir çalışma zarfı

Denge, hiçbir şeyin değişmediği, yani tüm türevlerin sıfır olduğu durumdur. İki rotor da kapalıyken yerçekimi tek başına (A − B)·cos alpha_v = C·sin alpha_v olduğunda dengelenir:

$$\alpha_{v,\text{dinlenme}} = \arctan\left(\frac{A-B}{C}\right) = \arctan\left(\frac{-0.01577}{0.01986}\right) = -0.671 \text{ rad } (-38.5^\circ)$$

Her simülasyon bu dinlenme durumundan başlar. Başka bir pitch açısını tutmak için ana rotor, yerçekiminin eksilttiği torku tam olarak sağlamalıdır. Bunu yapan sabit girişe **trim** girişi denir ve o açıda dS_v/dt = 0 çözülerek bulunur. Kod bunu `trim_main_input` içinde Brent yöntemiyle yapar.

Tutulan pitch	Trim girişi u_v
−0,4 rad (−23°)	0,40
−0,2 rad (−11,5°)	0,59

Tutulan pitch	Trim girişi u_v
0,0 rad (0°)	0,96
yaklaşık +0,09 rad'ın üzeri	u_v ⩽ 1 ile çözüm yok

Bu parametre setiyle kiriş yaklaşık +0,09 rad'ın üzerinde tutulamaz. Çalışmadaki tüm referansların −0,45 ile −0,05 rad arasında olmasının nedeni budur.

Tüm karşılaştırmayı açıklayan iki sayı

Pitch dinamiği −0,2 rad civarında doğrusallaştırıldığında yaklaşık 0,22 N·m/rad'lık bir yerçekimi "yay"ı sertliği elde edilir. J_v = 0,0554 kg·m² ile bu şu anlama gelir:

$$\omega_n = \sqrt{\frac{0.22}{0.0554}} \approx 2.0 \text{ rad/s}, \qquad \zeta = \frac{K_v}{2J_v\omega_n} = \frac{0.0095}{2 \times 0.0554 \times 2.0} \approx 0.04$$

0,04'lük bir sönüm oranı (zeta) son derece düşüktür: kendi haline bırakılan kiriş sayısız kez ileri geri salınır. Bunun üstüne, itkinin u_v'ye göre eğimi u_v = 0,4 civarında (pitch −0,4 rad), u_v = 0,6 civarına (pitch −0,2 rad) göre yaklaşık 1,9 kat büyüktür. Yani sistemin doğal sönümü çok azdır ve etkin kazancı çalışma noktasıyla değişir. Bölüm 1.2'de açıklandığı gibi, tek bir sabit PID kazanç seti her yerde ideal olamaz.

Bölüm 1.2 — PID kontrol

Bir PID kontrolcü girişi şimdiki hatadan, hatanın birikmiş geçmişinden ve hatanın şu anki eğiliminden hesaplar. Sistemin modeline ihtiyaç duymaz; bu onun en büyük gücüdür ve bu sistemde asıl zayıflığıdır.

Geri besleme ve takip hatası

Geri beslemeli kontrolde kontrolcü bir **referansı** r(t) (istenen açı) ölçülen **çıkış** y(t) (gerçek açı) ile karşılaştırır. Aradaki fark takip hatasıdır:

$$e(t) = r(t) - y(t)$$

Kontrolcünün görevi e(t)'yi hızlıca, büyük salınımlar olmadan ve eyleyicilerin sağlayabileceği girişlerle sıfıra götürmektir.

Sürekli zamanlı PID yasası

Ders kitabındaki PID yasası şöyledir:

$$u(t) = K_p\, e(t) \;+\; K_i \int_0^t e(\tau)\, d\tau \;+\; K_d\, \frac{de(t)}{dt}$$

Laplace bölgesinde (s karmaşık frekans değişkenidir) aynı kontrolcü bir transfer fonksiyonudur:

$$C(s) = \frac{U(s)}{E(s)} = K_p + \frac{K_i}{s} + K_d\, s$$

Her terimin ayrı bir fiziksel rolü vardır.

- **Oransal (P), Kp.** Şimdiki hatayla orantılı olarak iter, bir yay gibi. Daha büyük Kp daha hızlı yanıt verir, ancak bir noktadan sonra aşım ve salınım getirir.
- **İntegral (I), Ki.** Geçmiş hatayı biriktirir. Herhangi bir hata kaldıkça büyümeye devam eder, böylece kalıcı durum hatasını giderir. Pitch ekseninde bu zorunludur: −0,2 rad'ı tutmak yaklaşık 0,59'luk sabit bir u_v gerektirir ve sıfır hatada sabit bir çıkışı yalnızca integral sağlayabilir.
- **Türev (D), Kd.** Hatanın ne kadar hızlı değiştiğine tepki verir, bir sönümleyici veya amortisör gibi. Kirişin sahip olmadığı sönümü (zeta ≈ 0,04) sağlar.

Kodun gerçekte uyguladığı

Ders kitabı yasasının gerçek veya gerçekçi biçimde simüle edilmiş donanımda çalışması için dört pratik değişiklik gerekir. `trms/controllers/pid.py` dördünü de uygular.

1. Ölçüm üzerinden türev. Referanstaki bir basamakta de/dt bir an için sonsuz olur ve e'nin türevi çok büyük bir sıçrama üretir ("türev tekmesi"). Basamaklar arasında r sabit olduğu için orada de/dt = −dy/dt'dir; bu yüzden kod ölçümün türevini alır:

$$D(t) = -K_d \frac{dy(t)}{dt}$$

2. Türev filtresi. Saf türev alma yüksek frekanslı gürültüyü sınırsız büyütür. T_f = 0,1 s zaman sabitli birinci dereceden alçak geçiren bir filtre bunu sınırlar:

$$D(s) = -\frac{K_d\, s}{T_f\, s + 1}\, Y(s)$$

3. Ayrıklaştırma. Sayısal bir kontrolcü sabit örnekleme anlarında, t_k = k·Ts ve Ts = 0,05 s ile çalışır. Filtreye geri Euler yaklaşımı (dD/dt ≈ (D_k − D_k−1)/Ts), integrale de dikdörtgen kuralı uygulanınca `pid_step` içinde kullanılan güncelleme denklemleri elde edilir:

$$
\begin{aligned}
e_k &= r_k - y_k \\
P_k &= K_p\, e_k \\
I_k &= I_{k-1} + K_i\, T_s\, e_k \\
D_k &= \frac{T_f}{T_f + T_s}\, D_{k-1} \;-\; \frac{K_d}{T_f + T_s}\,\big(y_k - y_{k-1}\big) \\
v_k &= P_k + I_k + D_k \\
u_k &= \mathrm{sat}(v_k) = \min\big(\,\max(v_k,\,-1),\,1\big)
\end{aligned}
$$

Kod P_k, I_k ve D_k'yi `pid_P_*`, `pid_I_*`, `pid_D_*` olarak; doygunlaşmamış toplam v_k'yi `u_unsat_*` olarak; uygulanan u_k'yi de `u_pitch` ve `u_yaw` olarak kaydeder.

4. Anti-windup: koşullu integral ve integratör kıstırması. Çıkış ±1'de kırpıldığında sistem, kontrolcünün istediğinden daha azını alır. İntegral büyümeye devam eder ("windup") ve daha sonra geri sarılırken uzun ve büyük bir aşıma yol açar. Kod bunu iki katmanda engeller.

Katman 1, koşullu integral. Geçici bir integral ve çıkış hesaplanır:

$$
I_k^{\text{deneme}} = I_{k-1} + K_i\, T_s\, e_k, \qquad v_k^{\text{deneme}} = P_k + I_k^{\text{deneme}} + D_k
$$

Geçici çıkış bir sınırı aşacaksa *ve* hata o yönde daha da itiyorsa, integratör yalnızca çıkışı tam sınıra koyan değere kadar ilerler. Bu değerin ötesine hiç geçmez ve asla geri gitmez:

$$
I_k = \begin{cases}
\max\big(I_{k-1},\ \min(I_k^{\text{deneme}},\ u_{\max} - P_k - D_k)\big) & \text{eğer } v_k^{\text{deneme}} > u_{\max} \wedge e_k > 0 \\
\min\big(I_{k-1},\ \max(I_k^{\text{deneme}},\ u_{\min} - P_k - D_k)\big) & \text{eğer } v_k^{\text{deneme}} < u_{\min} \wedge e_k < 0 \\
I_k^{\text{deneme}} & \text{aksi halde}
\end{cases}
$$

Çıkış zaten sınırdaysa sınır değeri I_k−1'e eşittir ve integratör donar; bu, klasik koşullu integral kuralıdır. Kodun ilk sürümü, geçici çıkış bir sınırı aştığında yalnızca I_k−1'i koruyordu. Tek bir integral adımı sınırın ötesine sıçradığında integratör bir daha hiç

ilerlemiyor, büyük bir Ki ile de çıkış büyük bir hata altında 0'da takılıp kalabiliyordu. Yukarıdaki kural bunu düzeltmek için 14 Ekim 2026 kod incelemesinde getirildi.

Katman 2, integratör kıstırması. Katman 1, büyük ve ters yönlü bir P veya D terimi *toplamı* sınırlar içinde tutarken I'nın büyümesini durduramaz. Bu nedenle integral terimi tek başına da eyleyici aralığına kıstırılır:

$$I_k \leftarrow \min\left(\max(I_k,\ u_{\min}),\ u_{\max}\right)$$

`pid_integrator_frozen_pitch` ve `pid_integrator_frozen_yaw` bayrakları, katman 1'in integratörü sınırladığı örnekleri kaydeder (referans koşusunda pitch için 6 örnek).

Merkezi olmayan MIMO yapısı

Proje iki bağımsız tek girişli tek çıkışlı (SISO) döngü kullanır. Pitch hatası ana rotoru, yaw hatası kuyruk rotorunu sürer. Matris biçiminde kontrolcü köşegendir:

$$\begin{bmatrix} U_v(s) \\ U_h(s) \end{bmatrix} = \begin{bmatrix} C_v(s) & 0 \\ 0 & C_h(s) \end{bmatrix} \begin{bmatrix} E_v(s) \\ E_h(s) \end{bmatrix}$$

Köşegen dışındaki sıfırlar işin özüdür: hiçbir döngü diğerinin varlığından haberdar değildir. Otomatik ayarlanan kazançlar şunlardır:

Döngü	Kp	Ki	Kd
Pitch (u_v)	0,456	0,396	0,132
Yaw (u_h)	0,181	0,001	0,651

Yaw için Ki arama aralığının alt sınırındadır (10^{-3}); yani yaw döngüsü fiilen bir PD kontrolcüdür. Bu fiziksel olarak anlamlıdır: yaw ekseninde yenilmesi gereken bir yerçekimi yoktur, dolayısıyla dururken sabit bir girişe ve integral etkisine ihtiyaç yoktur.

PID, bağlanımlı ve doğrusal olmayan bir MIMO sistemde neden zorlanır

PID'nin buradaki zorluklarının dört ayrı nedeni vardır. İlk ikisi bu çalışmanın sonuçlarına hakimdir.

1. **Kazancı değişen bir sistemde sabit kazançlar.** İtkinin u_v'ye göre eğimi pitch −0,4 rad civarında, −0,2 rad civarına göre yaklaşık 1,9 kat büyüktür. Döngü kazancı, kontrolcü kazancı ile sistem kazancının çarpımıdır. Dolayısıyla −0,2 rad'da kararlılığın sınırında olan kazançlar −0,4 rad'da neredeyse iki kat döngü kazancı verir ve döngüyü kararlılık payının ötesine iter. Değerlendirmenin gösterdiği tam olarak budur: −0,2 rad'da sürekli bir salınım, t = 40 s'deki −0,4 rad basamağından sonra kararsızlığa dönüşür.
2. **Yavaş bir eyleyicinin arkasında çok düşük sönüm.** Tek sönüm kaynağı D terimidir. Ancak D terimi, 1,43 s'lik gecikmesi her düzeltmeyi geciktiren ana motor üzerinden etki eder. Kirişin doğal frekansında (yaklaşık 2 rad/s) bu gecikme yaklaşık 70° faz kaybına mal olur. Bu yüzden D terimi salınımı ancak kısmen bastırabilir. Daha fazla kazanç salınımı kötüleştirir, daha az kazanç yanıtı yavaşlatır. Optimizasyonun bulduğu en iyi uzlaşı bile salınır.
3. **Bozucu olarak görülen bağlanım.** Her döngü diğer ekseni yalnızca etkileri aracılığıyla görür. Yaw döngüsü kuyruk rotoru hızını değiştirdiğinde tepki momentumu pitch eksenini anında eğer. Pitch döngüsü ancak bir hata oluştuktan sonra düzeltme yapar; bu düzeltme ana rotor hızını değiştirir, bu da yaw'ı bozar. Genel olarak böyle döngüler birbirini uyarabilir ("kavga edebilir"). Bu koşuda PID'nin yaw döngüsü yavaş ve yumuşak olduğu için etkileşim hafiftir; baskın hata, 1. ve 2. nedenlerin yol açtığı pitch salınımıdır.
4. **Öngörüsüz sınırlar.** PID ±1 giriş sınırını hesaba katarak plan yapamaz. Anti-windup integralin durumu kötüleştirmesini engeller, ancak kontrolcü bir sınıra yaklaştığını sınıra ulaşana kadar bilmez.

Bunlar ayar hatası değildir. Pitch kazançları üzerinde 4.000 noktalı rastgele bir arama, optimizasyonun sonucundan daha iyi bir şey bulamadı (en iyi amaç değeri 20,8'e karşı 19,5). Alışılmış mühendislik çözümleri yerçekimi ileri beslemesi (trim girişini PID çıkışına eklemek) ve kazanç programlamadır (kazançları pitch açısıyla değiştirmek). Karşılaştırma saf PID ile N-MPC arasında kalsın diye ikisi de bilinçli olarak dışarıda bırakılmıştır.

Bölüm 1.3 — Doğrusal Olmayan Model Öngörülü Kontrol (N-MPC)

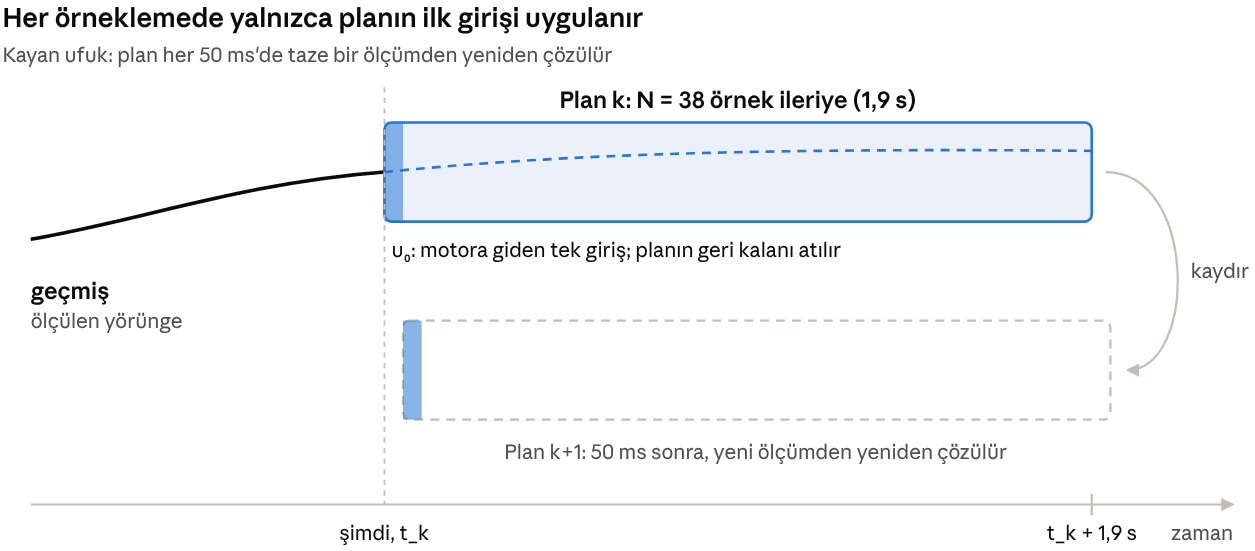
N-MPC sabit bir kontrol yasasının yerine her örneklemede yeniden çözülen bir optimizasyon koyar. Sistemin doğrusal olmayan modelini gelecekteki bir pencere boyunca simüle eder, bir maliyeti en aza indiren giriş dizisini seçer, yalnızca ilk girişi uygular ve 50 ms sonra işlemi tekrarlar.

Kayan ufuk ilkesi

Her t_k örnekleme anında kontrolcü dört adım uygular:

1. **Ölç:** sistemin şu anki x̂_k durumunu alır.
2. **Tahmin et ve optimize et:** modeli kullanarak, tüm kısıtlara uyarken maliyeti en aza indiren N gelecek girişlik u_0, …, u_N−1 dizisini bulur.
3. **Uygula:** yalnızca ilk girişi, u_0'ı, bir Ts örnekleme süresi boyunca sisteme uygular.
4. **Kaydır:** pencereyi bir adım ileri alır ve 1. adıma döner.

Pencere her zaman geleceğe aynı uzaklıkta baktığı için buna **kayan ufuk** (receding horizon) denir. Her adımda yeniden planlamak, açık çevrim bir planı geri beslemeye dönüştürür: model hataları ve bozucular bir sonraki ölçümde görünür ve bir sonraki plan bunları düzeltir.



kayan ufuk ilkesi · şema, çizilmiş veri değil

Planın tamamı 50 ms sonra taze bir ölçümden yeniden hesaplanır; bu yüzden motorlara yalnızca ilk girişi ulaşır.

Tahmin modeli

Optimizasyon ayrık zamanlı bir x_k+1 = F(x_k, u_k) modeline ihtiyaç duyar. Kod F'yi, Bölüm 1.1'deki sürekli f modelini klasik dördüncü dereceden Runge–Kutta (RK4) yöntemiyle integre ederek oluşturur. h adımıyla bir RK4 adımı şöyledir:

$$
\begin{aligned}
k_1 &= f(x,\, u) \\
k_2 &= f\left(x + \tfrac{h}{2} k_1,\, u\right) \\
k_3 &= f\left(x + \tfrac{h}{2} k_2,\, u\right) \\
k_4 &= f(x + h\, k_3,\, u) \\
x^{+} &= x + \tfrac{h}{6}\left(k_1 + 2k_2 + 2k_3 + k_4\right)
\end{aligned}
$$

Giriş her örnekleme boyunca sabit tutulur (“sıfırıncı derece tutucu”). Tahmin modeli örnekleme başına 2 RK4 adımı (h = 0,025 s) kullanır. Simüle edilen sistem örnekleme başına 10 adım (h = 0,005 s) kullanır ve “gerçek” sistem rolünü oynar. İkisi de tamamen aynı denklem kodunu kullandığından, ana karşılaştırmada tahmin modeli neredeyse mükemmeldir. Modelin hatalı olduğu durum Bölüm 5′teki dayanıklılık çalışmasında ayrıca incelenir.

Ufuk

Tahmin ufku N, kontrolcünün üzerinde plan yaptığı gelecek örnek sayısıdır. Ayarlanan değer N = 38′dir ve Ts = 0,05 s ile şu süreyi kapsar:

Image — Tahmin ufku uzunluğu

Ufuk, bir eylemin sonuçlarını görecek kadar uzun olmalıdır. Ana motor tek başına, komut edilen bir değişimin %63′ünü kat etmek için 1,43 s′ye ihtiyaç duyar ve kiriş yaklaşık 3 s periyotla salınır. Daha uzun bir ufuk daha uzağı görür, ancak her optimizasyonu daha büyük ve yavaş yapar. Ayarlayıcı bu yüzden N′yi 15 ile 40 arasında aradı.

Bölüm 5′teki **ufuk taraması** bu sınırı doğrudan ölçer. N ⩽ 20 (⩽ 1,0 s) ile kontrolcü eylemlerinin sonuçlarını yeterince uzağa kadar göremez ve döngü ıraksar: yaw kontrolden çıkar, pitch basamakları da oturmaz. N = 25 (1,25 s) ile döngü kararlı hale gelir. Ayarlanan N

= 38′in ötesinde ise kazanç küçüktür, çözüm süresi ise artmaya devam eder.

Maliyet fonksiyonu

J maliyeti planlanan bir yörüngenin tamamını puanlar. Bu projede takip hatasını, açısal hızları ve giriş *değişimlerini* cezalandırır:

Image — N-MPC maliyet fonksiyonu

Buradaki takip hataları ve giriş artışları şöyledir:

Image — Takip hataları, giriş artışları ve aşama ağırlıkları

Burada u_−1 bir önceki örneklemede gerçekten uygulanan giriştir; böylece ilk artış, eyleyicinin zaten yaptığına göre bir sıçramayı cezalandırır. Kısa matris gösteriminde:

Image — Matris biçiminde N-MPC maliyeti

Image — Ayarlanmış Q ve R ağırlıkları

Her ağırlık bir önceliği ifade eder.

- **Hatalar üzerindeki Q** (q_p = 1, ölçek olarak sabit; q_y = 3,05). Ayarlayıcı, rad² başına yaw hatasını pitch hatasının yaklaşık üç katı önemli buldu.
- **Hızlar üzerindeki Q** (0,0042 ve 0,343). Bunlar sönüm gibi davranır: kontrolcüyü hızlı açısal hareket üretmekten caydırır ve aşımı azaltır. Yaw hızı pitch hızından çok daha fazla cezalandırılır.
- **Artışlar üzerindeki R** (0,0128 ve 0,352). Sıçramalı girişleri caydırır. Küçük pitch değeri, kontrolcünün ana motor girişini sert biçimde değiştirebilmesine izin verir; daha büyük yaw değeri kuyruk girişini daha düzgün tutar.
- **Terminal ağırlığı** (w_N = 5). Son tahmin edilen durum beş kat sayılır. Bu, ufkun ötesindeki her şeyin maliyetine yaklaşır ve planı referansa yakın bitmeye zorlar.

u yerine Δu'yu cezalandırmak bilinçli bir seçimdir. Pitch'i tutmak büyük ve sabit bir u_v (yaklaşık 0,4–0,96) gerektirir. u'nun kendisine ceza vermek, kirişi havada tutmak için gereken girişle savaşır ve kalıcı durum hatası bırakırdı. Δu üzerindeki ceza ise giriş sabitlendikten sonra hiçbir şeye mal olmaz.

Kısıtlar

Kısıtlar, planın uymak zorunda olduğu kesin sınırlardır. MPC'nin bunları doğrudan içerebilmesi, PID'ye göre başlıca üstünlüklerinden biridir.

Giriş doygunluğu (burada kullanılır). Motor gerilimleri $-1 \leqslant u \leqslant 1$ ile sınırlıdır. PID çıkışını hesapladıktan *sonra* kırpar, dolayısıyla doygunlaştığı her an iç planı ile gerçeklik birbirini tutmaz. N-MPC sınırı optimizasyonun *içine* alır; yaptığı her plan, eyleyicilerin gerçekten uygulayabileceği bir plandır. Girişlerinin kısa süreler boyunca tam ±1'de durmasının nedeni budur: optimum sıklıkla sınır üzerindedir.

Durum kısıtları (burada kullanılmaz). Genel olarak bir MPC, tahmin edilen durumların güvenli bir kümede kalmasını da isteyebilir; bu g(x_k) ⩽ 0 biçiminde yazılır. Bir TRMS için bu bir pitch aralığı olabilir, örneğin −1 ⩽ alpha_v ⩽ 0,1. Uygulamada durum sınırları genellikle *yumuşak* yapılır: ağır cezalı bir $s \geqslant 0$ gevşek değişkeni eklenir, böylece sınır karşılanamadığında da problem çözülebilir kalır:

Image — Yumuşak durum kısıtı

Bu proje durum kısıtı koymaz, yalnızca giriş sınırları vardır. Durumların karar değişkeni sınırları `nmpc.py` içinde ±sonsuz olarak ayarlanır.

Tam optimizasyon problemi

Kontrolcü her örneklemede şu doğrusal olmayan programı (NLP) çözer:

Image — N-MPC doğrusal olmayan programı

Çoklu atış: problemin kuruluşu

Bu NLP iki şekilde kurulabilir. *Tekli atışta* (single shooting) yalnızca girişler değişkendir ve durumlar ileriye doğru simülasyonla hesaplanır. Bu kodun kullandığı **çoklu atışta** (multiple shooting) tahmin edilen durumlar da değişkendir ve dinamikler, çözücünün sağlattığı

eşitlik kısıtlarına dönüşür. Çoklu atış, kararsız veya az sönümlü bir sistem için sayısal olarak daha iyi koşullanmıştır: erken bir girişteki küçük bir değişiklik, tahmin edilen yörüngenin tamamını savurmaz.

N = 38 için problemin boyutları:

Büyüklük	Formül	Sayı
Durum değişkenleri	6 (N + 1)	234
Giriş değişkenleri	2 N	76
Toplam karar değişkeni w	6(N+1) + 2N	310
Eşitlik kısıtları (başlangıç + dinamik)	6 (N + 1)	234
Giriş sınırları	2 N	76
Parametreler p (durum, referanslar, önceki giriş)	6 + 2N + 2	84

IPOPT problemi nasıl çözer

IPOPT (Interior Point OPTimizer), büyük doğrusal olmayan programlar için bir çözücüdür. Sınırları, sınırlarda sonsuza giden ve iterasyonları kesinlikle içeride tutan logaritmik bir bariyer terimi ekleyerek ele alır:

Image — IPOPT bariyer problemi

Bu problemin optimallik (Karush–Kuhn–Tucker, KKT) koşullarını Newton yöntemiyle çözer, ardından bariyer parametresi mu'yu sıfıra doğru küçülterek çözümü gerçek kısıtlı optimuma yaklaştırır. KKT koşulları Lagrange fonksiyonunu kullanır:

Image — Lagrange fonksiyonu

Newton yöntemi J ve g'nin birinci ve ikinci türevlerine ihtiyaç duyar. CasADi bunları sonlu farklarla değil, **otomatik türev alma** ile tam olarak hesaplar; bu da her iterasyonu doğru ve hızlı yapar.

Sıcak başlatma. Ardışık problemler yalnızca bir örnek kadar farklıdır, bu yüzden önceki örneklemenin cevabı mükemmel bir ilk tahmindir. Kod önceki çözümü bir adım ileri kaydırır: tahmin edilen her durum ve giriş bir aşama öne taşınır, son aşama tekrarlanır ve ilk durum yeni ölçümle değiştirilir. Dinamik kısıtlarının ve giriş sınırlarının Lagrange çarpanları da aynı şekilde kaydırılır; böylece birincil ve dual tahminler aynı zaman adımlarını tanımlar. IPOPT ardından küçük bir bariyer parametresi (mu_init = 10^{-3}) ve uyarlamalı bir mu stratejisiyle başlar. Sıcak başlatma, kapalı çevrim koşu süresini soğuk başlatmaya göre kabaca yarıya indirdi.

İlk sürüm birincil tahmini kaydırıyor, ancak çarpanları kaydırmadan aktarıyordu. Kaydırmayı tutarlı hale getirmek (14 Ekim 2026 incelemesi), 60 s'lik değerlendirme koşusundaki IPOPT iterasyonlarını 2.075'ten 1.891'e (%9) düşürdü. Kapalı çevrim yörüngesi en fazla $1{,}1 \times 10^{-4}$ rad değişti; bu, çözücü toleransı düzeyindedir.

Çözücünün yeniden kullanımı. q ve r ağırlıkları NLP'ye sayısal sabitler olarak değil, p parametre vektörüne eklenen *parametreler* olarak girer. Bu yüzden ifade grafiği, tam türevleri ve IPOPT örneği yalnızca modele, Ts'ye, N'ye, terminal ağırlığına ve çözücü seçeneklerine bağlıdır. `_build_solver` bunları her kombinasyon için bir kez kurar ve önbelleğe alır. Yüzlerce ağırlık seti deneyen ayarlayıcı böylece yalnızca ufuk N değiştiğinde yeni bir çözücü kurar. Önceden her aday yaklaşık 1–3 s kurulum maliyeti ödüyordu.

N-MPC, PID'nin zorlandığı yerde neden başarılıdır

- **Doğrusal olmayan modeli kullanır.** Değişken itki eğimi, yerçekimi eğrisi ve motor gecikmesi tahminin içindedir; sabit kazançların yanlış yapabileceği bir şey kalmaz.
- **Bağlanımı bilir.** Tepki momentumu ve merkezkaç terimleri modeldedir; planlanan kuyruk girişleri tahmin edilen pitch sonuçlarıyla birlikte gelir. Yine de yaw basamaklarında 17°'ye kadar pitch sarsıntılarını kabul eder: q_y = 3,05 ile hızlı bir yaw dönüşü maliyet açısından sakin bir pitch ekseninden daha değerlidir ve referans önizlemesi olmadan basamak gelmeden hazırlığa başlayamaz.
- **Sınırlar içinde plan yapar.** Doygunluk bir sürpriz değil, bir kısıttır.
- **Zamanda ileriyi planlar.** Şimdi itkiyi azaltmanın daha sonra bir düşüşe yol açtığını görür ve buna göre davranır; yalnızca şimdiki hataya tepki veren bir PID bunu yapamaz.

Uyarılar

- **Tam durum.** Kontrolcüye altı durumun tümü tam olarak verilir. Donanımda yalnızca iki açı ölçülür; S_v, S_h ve motor durumlarını bir gözlemcinin (örneğin genişletilmiş Kalman filtresi veya hareketli ufuk kestiricisi) tahmin etmesi gerekir.

- **Model uyumsuzluğu.** Ana karşılaştırmada tahmin modeli ve sistem aynı denklemleri paylaşır. Bölüm 5'teki dayanıklılık çalışması bunun bedelini ölçer: çoğu tek parametre hatası zararsızdır, ancak itki ölçeğindeki %15'lik veya karşı ağırlıktaki %5'lik bir hata toplam IAE'yi 2–4 katına çıkarır.
- **Biçimsel kararlılık ispatı yok.** Terminal ağırlığı bir sezgiseldir. Titiz bir tasarım, özyinelemeli olurluluğu kanıtlanmış bir terminal kısıt kümesi ve terminal maliyeti eklerdi.
- **Hesaplama.** IPOPT 50 ms'lik örnekleme başına ortalama yaklaşık 11–16 ms harcadı; ancak aynı makinedeki farklı koşularda en yavaş örnek 110 ile 310 ms arasında sürdü ve örneklerin yaklaşık %1–1,5'i bütçeyi aştı. Gerçek zamanlı bir uygulama iterasyon sayısını sınırlar veya acados'takiler gibi bir gerçek zamanlı iterasyon (RTI) şeması kullanırdı.

Bölüm 1.4 — Otomatik ayarlama

Her iki kontrolcü de aynı global optimizasyon yöntemiyle, diferansiyel evrimle, aynı ayar senaryosunda aynı maliyeti en aza indirerek ayarlandı. PID 15.488 simülasyon koşusundan sonra J = 19,51'e, N-MPC ise 224 koşudan sonra J = 2,36'ya ulaştı.

Neden otomatik ve adil ayarlama

Bir karşılaştırma ancak her kontrolcü makul ölçüde olabileceği kadar iyi yapılmışsa anlamlıdır. Bir kontrolcüyü diğerinden daha özenli elle ayarlamak sonucu önceden belirlerdi. Bu nedenle proje ikisi için de tek bir amaç fonksiyonu, tek bir optimizasyon ailesi ve tek bir ayar senaryosu kullanır. Ardından onları *farklı* bir değerlendirme senaryosunda puanlar; böylece hiçbiri uydurulmuş olduğu yörüngenin aynısı üzerinden yargılanmaz.

Senaryo	Süre	Pitch basamakları (rad)	Yaw basamakları (rad)
Ayar	38 s	0 s'de −0,30; 12 s'de −0,10; 22 s'de −0,45	6 s'de +0,40; 18 s'de −0,30
Değerlendirme	60 s	0 s'de −0,20; 20 s'de −0,05; 40 s'de −0,40	10 s'de +0,50; 30 s'de −0,50; 50 s'de 0,00

Her iki senaryo da dinlenme durumundan (pitch −0,671 rad) başlar. Ayar senaryosu değerlendirmeyle aynı pitch aralığını kapsar ve 16 s'lik bir bekleme ile biter; bu bekleme yavaş kaymayı ve büyüyen salınımı ortaya çıkarır.

Ayar amaç fonksiyonu

`metrics.tuning_objective` içinde uygulanan J maliyet fonksiyonu şöyledir:

Image — Ayar amaç fonksiyonu J

Burada tau_k, iki eksenden herhangi birindeki en son referans değişikliğinden bu yana geçen süredir:

Image — Son referans değişikliğinden bu yana geçen süre

Iraksama cezası ise şöyledir:

Image — Iraksama cezası

Terimler dört şeyi ödüllendirir: küçük hata (ITAE), düzgün girişler (u′nun toplam varyasyonu), sınırlı aşım (%5′in üzerindeki her aşım cezalandırılır) ve kararlılık (P_div).

Neden IAE yerine ITAE

Düz IAE bir hatayı ne zaman oluşursa oluşsun eşit tartar. ITAE ise her hatayı son basamaktan bu yana geçen süreyle çarpar; süren bir hata giderek daha pahalıya mal olur. Sönümlenen hızlı bir geçici durum ucuzdur; yavaş bir kayma veya sürekli bir salınım pahalıdır.

Proje bunu zor yoldan öğrendi. İlk sürüm düz IAE kullanıyor ve ıraksamayı yalnızca pitch üzerinde denetliyordu. Optimizasyon bu durumda sıfıra yakın yaw kazançları (Kp ≈ 0,01) seçti. Bunların yavaş kayması kısa ayar penceresinde gizli kaldı, ancak daha uzun değerlendirme koşusunda yaklaşık 2.000° yaw′a kaçtı. ITAE′ye geçmek, yaw ıraksama denetimini eklemek ve son beklemeyi uzatmak bunu düzeltti.

Diferansiyel evrim

Diferansiyel evrim (DE), popülasyon tabanlı ve türev gerektirmeyen bir global optimizasyon yöntemidir. Burada iyi bir seçimdir, çünkü J′nin biçimi düzensizdir: koşuların ıraksadığı yerlerde süreksizdir, çok sayıda yerel minimumu vardır ve gradyanı yoktur. Her

aday bir θ parametre vektörüdür. Optimizasyon bir aday popülasyonu tutar ve onu nesilden nesile iyileştirir.

g neslinin her x_i üyesi için SciPy'ın varsayılan `best1bin` stratejisi üç şey yapar.

1. Mutasyon. Mevcut en iyi üyeden ve iki rastgele üyenin, r1 ile r2'nin, farkından bir mutant oluşturulur:

Image — Diferansiyel evrim mutasyonu

2. Binom çaprazlama. Mutant, mevcut üyeyle koordinat koordinat, CR = 0,7 çaprazlama olasılığıyla karıştırılır. Rastgele bir j_rand koordinatı her zaman mutanttan alınır:

Image — Binom çaprazlama

3. Açgözlü seçim. Hangisi daha iyiyse o tutulur:

Image — Açgözlü seçim

Başlangıç popülasyonu, noktaları arama kutusuna eşit dağıtan düşük tutarsızlıklı bir küme olan Sobol dizisidir. SciPy bir Sobol popülasyonunu 2'nin bir kuvvetine yuvarlar; aşağıdaki nesil başına 128 ve 32 koşu sayılarının nedeni budur. Güncelleme "ertelenmiş" (deferred) biçimdedir: herhangi bir üye değiştirilmeden önce tüm nesil değerlendirilir; bu da toplu ve paralel değerlendirmeye izin verir. Tohum 42'ye sabitlenmiştir, dolayısıyla ayarlama tekrarlanabilirdir.

Arama uzayları

Kazançlar ve ağırlıklar 10 tabanında logaritmaları üzerinden aranır. Bir kazancın doğru değeri 0,001 ile 10 arasında herhangi bir yerde olabilir; logaritmik ölçek her onluk basamağa eşit ilgi gösterir.

Kontrolcü	Boyut	Aranan parametreler (log10 sınırları)
PID	6	Kp ∈ [−2; 1,5], Ki ∈ [−3; 1], Kd ∈ [−3; 1], pitch ve yaw için ayrı ayrı

Kontrolcü	Boyut	Aranan parametreler (log10 sınırları)
N-MPC	6	q_y ∈ [−1; 1,5]; q_dp, q_dy, r_p, r_y ∈ [−4; 0]; tamsayı N ∈ [15; 40] (doğrusal)

N-MPC için q_p 1'e sabitlenmiştir. Tüm ağırlıkları aynı sabitle çarpmak optimal planı değiştirmez; bu fazlalığı gidermek için bir ağırlığın sabitlenmesi gerekir. Ufuk N bir tamsayıdır; SciPy'ın `integrality` seçeneği bu koordinatı yuvarlar.

Ayar koşuları

	PID	N-MPC
Nesil başına aday	128	32
Nesil sayısı (başlangıç dahil)	121	7
Simülasyon koşusu	15.488	224
Değerlendirme stratejisi	tüm nesil tek bir vektörel simülasyonda (numpy PID, CasADi Function.map sistemi)	aday başına bir kapalı çevrim IPOPT koşusu, paralel süreçlerde, ufuk başına bir önbellekli çözücü
Duvar saati süresi (2 çekirdekli makine)	519 s	2.039 s
Son amaç değeri J	19,51	2,36

İki bütçe arasında 70 kat fark vardır, çünkü tek bir N-MPC koşusu tek bir PID koşusundan çok daha pahalıdır. Her N-MPC koşusu yaklaşık 760 doğrusal olmayan program çözer (örnek başına bir tane); PID'nin tüm popülasyonu ise tek bir dizi işlemiyle ilerler. İki arama da bir yakınsama toleransında değil, iterasyon sınırında durdu. Daha fazla nesil her iki sonucu da biraz iyileştirebilir.

Yukarıdaki duvar saati süreleri, önbellekteki parametreleri üreten ilk koda aittir. 14 Ekim 2026 optimizasyonundan sonra (Bölüm 2, *Sayısal güvenceler ve performans*) aynı aramalar aynı ayarlar ve tohumlarla PID için 142 s, N-MPC için 1.181 s sürer. %0,3 içinde aynı optimumlara, J = 19,51 ve 2,36'ya ulaşırlar. Önbellekteki parametreler korunmuştur ve yeni kodla tam olarak eskisi gibi puan alırlar (19,507 ve 2,363).

DE bir sezgiseldir ve global optimumu garanti etmez. Burada PID sonucu bağımsız olarak denetlendi: 4.000 rastgele pitch kazanç setinin hepsi daha kötü puan aldı. N-MPC'nin ayarlanan ufku ise Bölüm 5'teki ufuk taramasıyla ayrıca doğrulanır.

Bölüm 2 — Kod tabanının mimarisi ve çalışması

Depo, tek bir giriş betiği `main.py` tarafından sürülen küçük bir Python paketidir, `trms/`. Son eklemelerle birlikte bunun yanında bir de `studies/` paketi vardır: ana kodu değiştirmeden onun üzerinde çalışan N-MPC çalışmaları ve 3B web simülasyonu. Tüm fizik tek bir fonksiyonda, `model.terms` içindedir ve diğer her parça onu yeniden kullanır: simüle edilen sistem, N-MPC'nin tahmin modeli, trim hesabı ve çalışmalardaki bozulmuş sistemler.

Depo yerleşimi

```
TRMS-Kontrol/
├── main.py                     giriş noktası: ayarla → simüle et → kaydet →
çiz → karşılaştır
├── requirements.txt            numpy, scipy, pandas, matplotlib, openpyxl,
pyyaml, casadi
├── config/
│   ├── settings.yaml           tüm deney ayarları
│   └── tuned_parameters.json   ayarlanmış kazanç ve ağırlıkların önbelleği
+ ayar raporları
├── trms/
│   ├── params.py               fiziksel parametreler (oluşturulurken
doğrulanır)
│   ├── model.py                doğrusal olmayan dinamik, derlenmiş sistem,
RK4, trim, zarf
│   ├── scenario.py             referans sinyalleri ve basamak bölümleme
│   ├── controllers/
│   │   ├── pid.py              iki katmanlı anti-windup'lı ayrık PID
│   │   └── nmpc.py             CasADi/IPOPT N-MPC, ufuk başına önbellekli
çözücü
│   ├── tuning.py               diferansiyel evrim ayarlayıcıları
│   ├── cache.py                imza denetimli JSON önbelleği (atomik yazma)
│   ├── simulation.py           kapalı çevrim simülasyonu, önceden ayrılmış
telemetri
│   ├── metrics.py              metrikler ve ayar amaç fonksiyonu
│   ├── plotting.py             grafikler
│   └── reporting.py            CSV / JSON / NPZ / Excel / Markdown
```

```
yazıcıları
├── studies/                    YENİ: ana koşunun üzerine kurulan çalışmalar
|   ├── common.py               ortak kurulum, paralel N-MPC koşuları,
ıraksama denetimi
|   ├── horizon_sweep.py        N-MPC ufuk taraması
|   ├── robustness.py           model uyumsuzluğuna dayanıklılık
|   ├── animate_3d.py           bir koşunun 3B web tekrarı (trms_3d.html)
|   ├── templates/trms_3d.html  3B sayfanın şablonu
|   └── vendor/three.min.js     gömülü three.js (çevrimdışı çalışır)
├── tests/test_core.py          regresyon testleri (python
tests/test_core.py)
└── docs/results/               referans koşunun grafikleri ve özeti
    └── studies/                YENİ: çalışmaların yayımlanmış sonuçları
```

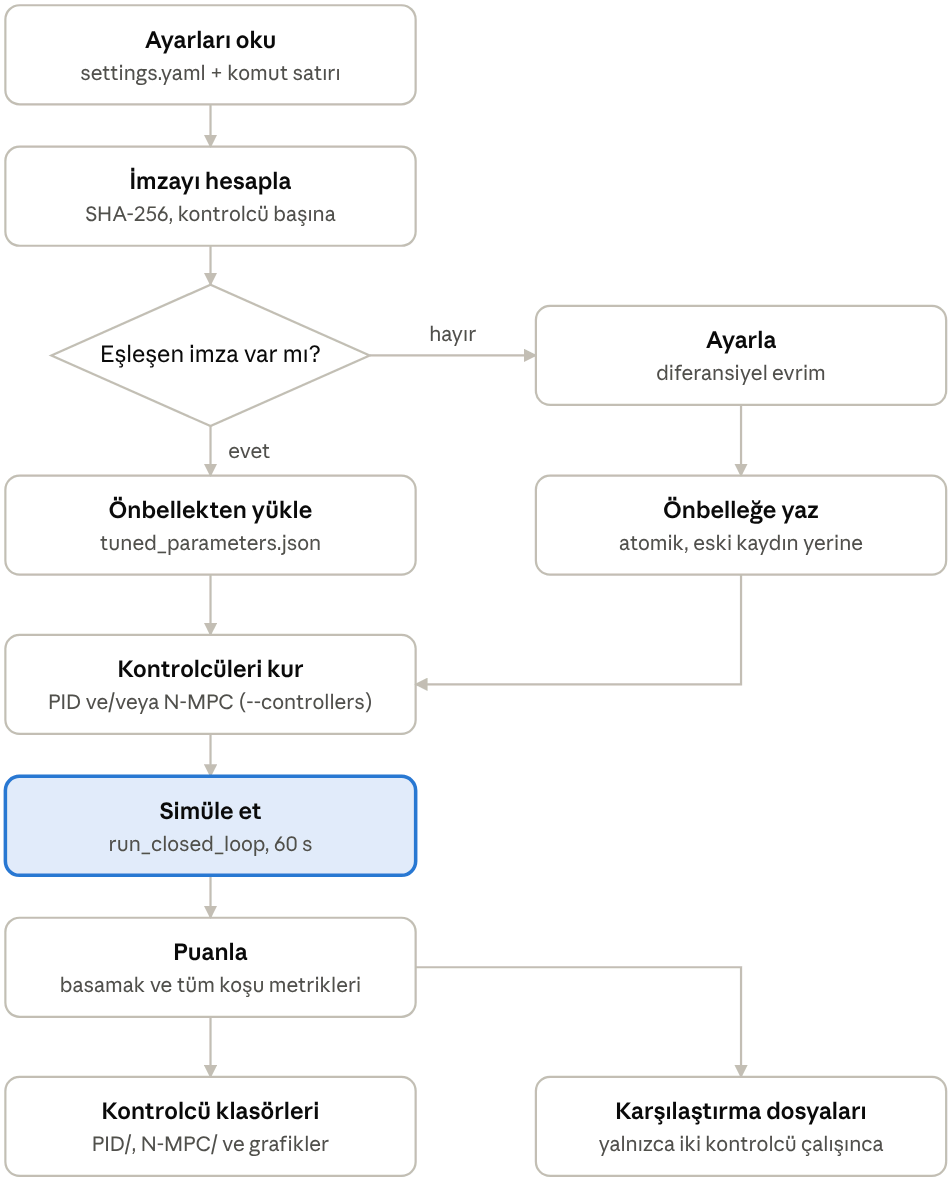
Bir koşu nasıl çalışır

`python main.py` şu adımları sırayla uygular:

1. **Argümanları ayrıştırır.** `--retune {pid,nmpc,all}` yeniden ayarlamayı zorlar; `--quick` arama bütçesini küçültür ve ayrı bir önbellek dosyası kullanır; `--settings`, `--cache` ve `--output-root` yolları geçersiz kılar. **Yeni:** `--controllers pid`, `--controllers nmpc` veya ikisi birden (varsayılan) hangi kontrolcülerin çalışacağını seçer.
2. **Ayarları yükler:** `config/settings.yaml` dosyasından; parametre setini, dinlenme başlangıç durumunu ve iki senaryoyu oluşturur.
3. **Her kontrolcü için önbelleği denetler.** Ayarlamayı etkileyen her şeyin imzasını hesaplar. Önbellekteki bir kayıt aynı imzaya sahipse ve yeniden ayarlama zorlanmadıysa onu kullanır. Aksi halde ayarlayıcıyı çalıştırır ve sonucu saklar.
4. **Kontrolcüleri kurar:** ayarlanmış parametrelerden.
5. **Simüle eder:** her kontrolcüyü 60 s'lik değerlendirme senaryosunda `run_closed_loop` ile; bu bir telemetri tablosu döndürür.
6. **Puanlar:** her koşuyu basamak başına ve tüm koşu metrikleriyle.
7. **Her kontrolcünün klasörünü yazar** (`PID/`, `N-MPC/`): telemetri, metrikler, parametreler, grafikler.
8. **Karşılaştırma dosyalarını yazar:** koşunun kök klasörüne; birleşik zaman serileri, metrik tabloları, bir Excel çalışma kitabı, üst üste grafikler ve `summary.md`. **Yeni:** bu dosyalar yalnızca iki kontrolcü birlikte çalıştığında yazılır. Tek kontrolcülü bir koşuda kökte o kontrolcünün sonuçlarını içeren tek sütunlu bir `summary.md` oluşur.
9. **Yazdırır:** özet tabloyu ve çıktı yolunu.

Eşleşen bir imza ayarlamayı tamamen atlar

python main.py veri akışı; önbellek denetimi her kontrolcü için ayrı yapılır



Tek kontrolcülü koşuda kökte yalnızca tek sütunlu summary.md yazılır.

main.py veri akışı · depodaki koda göre

Önbellek denetimi tek dallanmadır: eşleşen bir imza ayarlamayı tamamen atlar. Ayarlayıcı da her adayı puanlamak için modeli ve simülasyonu kendi içinde çalıştırır.

Dosya dosya inceleme

`main.py` — yalnızca orkestrasyon; fizik veya matematik içermez. Modülleri yukarıdaki sırayla birbirine bağlar ve her koşu klasörüne zaman damgası verir (`outputs/run_YYYYmmdd-HHMMSS/`); böylece koşular birbirinin üzerine yazmaz. Koşuda kullanılan ayarları `settings_used.yaml` olarak kaydeder. **Yeni:** tüm metin dosyaları açıkça UTF-8 ile okunur ve yazılır; bu, Windows'ta "°" veya "→" gibi karakterlerde yaşanan çökmeyi giderir.

`config/settings.yaml` — deneyi değiştirmenin tek yeri. Bölümleri:

Bölüm	Kontrol ettiği
`simulation`	örnekleme süresi Ts = 0,05 s, sistem RK4 alt adımları (10)
`scenarios`	değerlendirme ve ayar basamak dizileri, [eksen, zaman, değer] olarak
`objective`	ortak ayar maliyetinin ağırlıkları ve sınırları
`pid`	türev filtresi T_f, log10 arama sınırları, DE bütçesi
`nmpc`	sabit q_p, terminal ağırlığı, tahmin alt adımları, referans önizlemesi, IPOPT seçenekleri, arama sınırları, DE bütçesi
`quick`	`--quick` için küçültülmüş DE bütçeleri
`output`	çıktı kök klasörü

`trms/params.py` — `TRMSParams`, her fiziksel sabiti ve polinomu tutan dondurulmuş (değiştirilemez) bir dataclass. Değiştirilemez olması, hiçbir kodun koşu ortasında bir parametreyi yanlışlıkla değiştirememesi demektir; ayrıca nesneyi hashlenebilir yapar, böylece çözücü ve integratör önbelleklerinde anahtar olarak kullanılabilir. Oluşturulurken kendini doğrular. Modeldeki her bölen pozitif kalmalıdır: J_v, T_mr, T_tr ve J_h(alpha_v) = D cos² + E sin² + F; bu sonuncusu F + min(D, E) > 0 gerektirir. Sürtünme ve eylemsizlikler negatif olmamalı, u_min < u_max olmalıdır. Geçersiz bir set, bir simülasyonun derinliklerinde sıfıra bölme üretmek yerine açık bir hata verir. `to_dict()` onu önbellek imzası için serileştirir; `rest_pitch` dinlenme açısını `atan2(A - B, C)` ile hesaplar; bu

sıfıra bölemez ve C'nin her iki işareti için de kararlı kökü seçer. Dosya ayrıca kayıt sütun adları olarak kullanılan durum adlarını tanımlar. Dayanıklılık çalışması bozulmuş sistemleri `dataclasses.replace` ile bu sınıftan türetir; doğrulama onlara da uygulanır.

`trms/model.py` — fizik.

- `polyval` bir polinomu Horner kuralıyla değerlendirir. Düz sayılar, numpy dizileri ve CasADi sembolleri üzerinde aynı şekilde çalışır.
- `terms(x, u, p, sin, cos)` projenin kalbidir. Altı türevi ve her ara büyüklüğün (rotor hızları, itkiler, her tork, J_h) bir sözlüğünü döndürür. Trigonometrik fonksiyonlar argüman olarak verilir. `np.sin` ile çağrıldığında sayı hesaplar; `ca.sin` ile çağrıldığında CasADi için sembolik bir ifade grafiği kurar. Böylece tek bir fonksiyon hem sistemi hem kontrolcünün modelini tanımlar ve ikisi birbirinden uzaklaşamaz.
- `f_np` ve `aux_np` numpy sarmalayıcılarıdır. (6, T) biçiminde durumları kabul ederler; simülasyon telemetriyi oluşturmak için onları tüm yörünge üzerinde bir kez değerlendirir.
- `rk4_np` sıfırıncı derece tutucu girişle RK4 integrasyonu yapar. Testlerin karşılaştırdığı numpy referans uygulamasıdır.
- `rk4_casadi` iki CasADi `Function` nesnesi kurar: `F(x, u)`, bir örnekleme boyunca RK4 adımı; ve maliyet için Omega_v ile Omega_h'yi döndüren `rates(x)`. `functools.lru_cache` ile önbelleğe alınır; aynı istekler önceden kurulmuş grafiği yeniden kullanır.
- `Plant` simüle edilen sistemdir: 10 alt adımlı `rk4_casadi`, CasADi'nin SX sanal makinesiyle değerlendirilir. Bir örnek numpy sürümündeki yaklaşık 3 ms yerine yaklaşık 40 µs sürer ve sonuç bit düzeyinde aynıdır. (6, B) biçiminde bir toplu girdi için `Function.map` kullanır; sütunlar üzerinde derlenmiş tek bir döngü.
- `rest_state`, `trim_main_input` (SciPy `brentq` kök bulma) ve `pitch_envelope` Bölüm 1.1'deki dengeleri hesaplar.

`trms/scenario.py` — `Scenario`, YAML basamak listesini bir r(t) referans fonksiyonuna dönüştürür. `steps()` zaman eksenini her referans değişikliği için bir bölüme ayırır. Her bölüm *herhangi bir* eksendeki bir sonraki değişiklikte biter; böylece bir basamağın metrikleri, diğer eksendeki sonraki bir basamağın bağlanım bozucusuyla kirlenmez.

`trms/controllers/pid.py` — `pid_step`, Bölüm 1.2'deki ayrık PID'yi iki anti-windup katmanıyla birlikte (2, B) biçimindeki diziler üzerinde uygular: B kazanç seti için aynı anda iki döngü. `PIDController` onu kayıtlı simülasyon için B = 1 ile sarar; böylece ayarlayıcı ve

son koşu aynı aritmetiği kullanır. `check_timing`, T_f + Ts bölenini sıfırlayabilecek tek durumlar olan Ts ⩽ 0 ve T_f < 0'ı reddeder; sonlu olmayan kazançlar da kurulumda reddedilir.

`trms/controllers/nmpc.py` — `_build_solver` Bölüm 1.3'teki çoklu atış NLP'sini kurar. X (6 × (N+1)) ve U (2 × N) için CasADi `SX` sembolleri oluşturur, dinamik kısıtlarını `F` ile zincirler, maliyeti biriktirir ve her şeyi `ca.nlpsol` ile bir IPOPT çözücüsünde derler. Parametre vektörü mevcut durumu, referansları, önceki girişi ve altı ağırlığı taşır; sonuç yalnızca (model, Ts, N, terminal ağırlığı, alt adımlar, seçenekler) üzerine bağlıdır ve önbelleğe alınır. `NMPCController` kurulumda önbellekteki çözücüyü alır. Her `step()` çağrısı yalnızca sayıları doldurur: parametre vektörü, kaydırılmış birincil tahmin ve kaydırılmış çarpanlar. Çözücüyü çağırır, ilk girişi uygular ve tahmin grafiği için tüm tahmin edilen yörüngeyi önceden ayrılmış dizilere yazar. Bir çözüm sonlu olmayan sayılar döndürürse kontrolcü önceki planını bir adım daha izler ve kaydırır; tekrarlanan hatalarda o plan boyunca ilerlemeye devam eder.

`trms/simulation.py` — `run_closed_loop` sanal laboratuvardır. Döngüden önce tüm referans sinyalini bir kerede değerlendirir ve her tamponu önceden ayırır: durumlar (T × 6), girişler (T × 2) ve her kontrolcü kayıt girdisi için tipli bir sütun. Döngü içinde yalnızca sıralı yapılması gerekeni yapar: kontrolcüden bir giriş ister, durumu ve girişi saklar, `Plant`'i bir örnek ilerletir. Durum sonlu olmaktan çıkarsa erken durur. Döngüden sonra `_telemetry_frame` her türetilmiş büyüklüğü tüm (6, T) yörünge üzerinde tek bir vektörel geçişte hesaplar: türevler, rotor hızları, torklar, doygunluk bayrakları ve `cumsum` ile birikimli integraller. `simulate_pid_batch` hızlı ayarlama yoludur: tüm bir PID popülasyonunu örnek başına tek bir `Function.map` sistem çağrısıyla ilerletir ve açık bir `diverged` maskesi tutar; durumu sonlu olmaktan çıkan bir üye işaretlenir ve dondurulur, böylece NaN amaç fonksiyonuna yayılmaz.

`trms/metrics.py` — `step_metrics` (bir referans basamağı), `global_metrics` (tüm koşu) ve `tuning_objective` (Bölüm 1.4'teki J maliyeti, bir popülasyon üzerinde vektörel). Bölüm 3 her metriği tanımlar. İkiden az sonlu örnek içeren bir basamak penceresi, örneğin erken durmuş bir koşudan sonraki, çökme yerine NaN metrikler üretir. Amaç fonksiyonunda sonlu olmayan bir puan veya ıraksamış olarak işaretlenmiş bir aday her zaman ıraksama cezasını alır.

`trms/tuning.py` — `tune_pid` ve `tune_nmpc`, `scipy.optimize.differential_evolution`'ı çağırır. N-MPC için her aday ayrı bir süreçte çalışmalıdır; bu yüzden `_nmpc_objective` en üst düzeyde tanımlı bir fonksiyondur ve yalnızca düz, pickle edilebilir veri (`ctx`) alır. Her işçi süreç kendi çözücü önbelleğini tutar;

bir ufuk N ile ilk karşılaştığında bir çözücü kurar ve o N'ye sahip sonraki her aday için yeniden kullanır. `run_nmpc_scenario` bir `diverged` bayrağı döndürür: pitch'i ıraksama sınırını geçen veya durumu sonlu olmaktan çıkan bir aday her zaman cezalandırılır.

`trms/cache.py` — `signature()` ve `ParameterCache`; aşağıya bakın.

`trms/plotting.py` — tüm grafikler, matplotlib'in etkileşimsiz `Agg` arka ucuyla çizilir; program ekranı olmayan sunucularda da çalışır. Renkler her grafikte kontrolcü başına sabittir (PID için mavi, N-MPC için turuncu) ve referans her zaman kesikli gri bir çizgidir. Çalışmalar aynı renkleri ve kaydetme yardımcılarını kullanır.

`trms/reporting.py` — telemetriyi dosyalara dönüştürür: basamak başına ve genel metrik tabloları, JSON, N-MPC tahminlerinin sıkıştırılmış NPZ dosyası, Excel çalışma kitabı ve `summary.md`. **Yeni:** basamak tablosu ve özet artık herhangi bir sayıda kontrolcüyle, tek bir kontrolcü dahil, çalışır.

`studies/common.py` — **yeni.** Çalışmaların ortak altyapısı. `load_setup()` ayarları, parametreleri, senaryoyu ve önbellekteki ayarlanmış N-MPC'yi ana koşudaki gibi yükler. `run_nmpc(p_model, p_plant, N, weights, cfg)` tek bir kapalı çevrim koşusu yapar; kontrolcü `p_model` ile tahmin yaparken sistem `p_plant` ile simüle edilir. Bir koşu erken durursa, |pitch| > 1,4 rad olursa veya |yaw hatası| > 1,5 rad olursa ıraksamış sayılır. `run_many` koşuları bir `ProcessPoolExecutor` ile paralel çalıştırır; `output_dir` sonuçları `outputs/studies/<çalışma>_<damga>/` altına yazar; `markdown_table` okunabilir tablolar üretir.

`studies/horizon_sweep.py`, **`studies/robustness.py`** ve **`studies/animate_3d.py`** — **yeni.** Bölüm 5'te ayrıntılı olarak anlatılır.

Kontrolcü arayüzü

Simülasyon hangi kontrolcüyü çalıştırdığını hiç denetlemez. Üç metodu olan herhangi bir nesne çalışır:

Metot	Amacı
`reset(x0, n_steps)`	iç belleği temizler (PID integratörü ve filtresi; N-MPC sıcak başlatması) ve `n_steps` örnek için kayıtları önceden ayırır
`step(t, x, ref_now, ref_fn)`	u girişini ve kaydedilecek iç değerlerin bir sözlüğünü döndürür

Metot	Amacı
`describe()`	çıktı JSON'u için kontrolcünün ayarlarını döndürür

Bu "ördek tipli" (duck-typed) arayüz, kazanç programlamalı bir PID gibi üçüncü bir kontrolcü eklemeyi tek bir yeni sınıf yazma meselesine indirger.

Dış kütüphaneler matematiği nasıl yapar

Kütüphane	Kullanım amacı	Yeri
NumPy	vektörel telemetri, PID aritmetiği, metrikler	her yerde
CasADi	sembolik model, otomatik türev alma, NLP kurulumu, derlenmiş sistem simülasyonu (`Plant`, `Function.map`)	`model.py`, `nmpc.py`
IPOPT (CasADi ile birlikte gelir)	her örneklemede NLP'nin çözümü	`ca.nlpsol('nmpc', 'ipopt', …)`
SciPy	diferansiyel evrim (ayarlama), Brent yöntemi (trim)	`tuning.py`, `model.py`
pandas	telemetri tabloları, CSV ve Excel yazımı, pivotlar	`simulation.py`, `reporting.py`, `studies/`
matplotlib	tüm grafikler	`plotting.py`, `studies/`
openpyxl	pandas'ın arkasındaki Excel motoru	`reporting.py`
PyYAML	ayarların okunması	`main.py`
hashlib, json, tempfile	önbellek imzaları, dosyalar ve atomik yazma	`cache.py`
concurrent.futures	çalışmalardaki paralel koşular	`studies/common.py`
three.js (gömülü, tarayıcıda çalışır)	3B web simülasyonu	`studies/vendor/`, `studies/templates/`

CasADi iki aşamada çalışır. **Kurulum** bir kez yapılır: `ca.SX.sym('X', 6, 39)` gibi semboller bilinmeyen sayıları temsil eder; onlar üzerindeki aritmetik tüm NLP'nin bir ifade grafiğini kurar; `nlpsol` da tam gradyan, Jakobiyen ve Hessian grafiklerini otomatik türev almayla türetir. `expand: True` ile bu grafikler hızlı skaler koda düzleştirilir. **Çözüm** her koşuda 1.200 kez yapılır: derlenmiş çözücü sayıları alır ve IPOPT, CasADi'nin kurduğu türev fonksiyonlarını kullanarak iterasyon yapar. Hiçbir türev sayısal olarak yaklaşıklanmaz.

Parametre önbelleği

Ayarlama dakikalardan onlarca dakikaya kadar sürer; bu yüzden sonuçları kaydedilir ve yeniden kullanılır. Zorluk, kaydedilmiş bir sonucun ne zaman hâlâ geçerli olduğunu bilmektir. Farklı bir model veya örnekleme süresi için ayarlanmış PID kazançlarını yeniden kullanmak, karşılaştırmayı sessizce geçersiz kılardı.

Önbellek bunu bir **imza** ile çözer: bir ayar sonucunu değiştirebilecek her şeyin SHA-256 özeti.

Image — Önbellek imzası

Kontrolcü bölümü arama sınırlarını ve DE bütçesini içerir. JSON anahtarları sıralanır; böylece eşit ayarlar her zaman aynı metni ve dolayısıyla aynı özeti üretir. Girdilerin herhangi bir yerinde tek bir basamağı değiştirmek bile tamamen farklı bir imza üretir.

Her koşuda, her kontrolcü için:

1. mevcut imza hesaplanır;
2. `tuned_parameters.json` bu imzaya sahip bir kayıt tutuyorsa ve `--retune` bu kontrolcüyü adlandırmadıysa kayıt yüklenir;
3. aksi halde ayarlama yapılır ve yeni kayıt eskisinin yerine yazılır.

`--quick` ile yapılan koşular ayrı bir dosya, `config/tuned_parameters_quick.json`, kullanır; böylece bir duman testi tam bütçeli sonuçların üzerine asla yazmaz. JSON dosyasının yapısı Bölüm 3'te anlatılır.

Önbellek dosyası atomik olarak yazılır: yeni JSON aynı klasördeki geçici bir dosyaya gider, sonra tek bir yeniden adlandırmayla önbelleğin yerini alır. Kesintiye uğrayan bir koşu (Ctrl-C, çökme, dolu disk) bu yüzden hiçbir zaman ayarlanmış tüm sonuçları sessizce silecek yarım yazılmış bir dosya bırakamaz. Çalışmalar önbelleği yalnızca okur; asla yeniden ayarlama yapmaz.

Sayısal güvenceler ve performans (14 Ekim 2026 incelemesi)

14 Ekim 2026'daki bir kod incelemesi dört mantık hatasını düzeltti ve mimariyi, kütüphaneleri veya çıktı dosyalarını değiştirmeden işlem hattını aşama başına 1,4–30 kat hızlandırdı. Önbellekteki parametrelerle PID telemetrisi anti-windup düzeltmesinden önce bit düzeyinde aynıydı; N-MPC yörüngesi en fazla $1{,}1 \times 10^{-4}$ rad değişti.

Hata	Etkisi	Düzeltme
Sonlu olmayan ayar puanı cezasız 0 ile değiştiriliyordu	bir NaN aday optimizasyonu kazanabilirdi	sonlu olmayan J veya bir `diverged` bayrağı her zaman 1.000 ceza ekler
Bir adım sınırı geçince PID integratörü donuyordu	çıkış doygunluğa ulaşamayabilir, büyük bir Ki ile 0'da kalabilirdi	integratör tam sınıra kadar ilerler, sonra kıstırılır (Bölüm 1.2)
N-MPC ayarlayıcısı sınır dışı örneği saklamadan duruyordu	ıraksayan bir aday cezadan kaçıyordu	aday ıraksamış olarak işaretlenir
Erken durmuş bir koşu üzerindeki metrikler	raporlama çöküyor ve tüm çıktılar kayboluyordu	kısa pencereler NaN ve `settled = False` raporlar

Daha küçük düzeltmeler:

- karşılaştırma zaman serisi artık iki koşuyu da kısa olanın uzunluğuna kesmez;
- doygunluk gerçek eyleyici sınırlarına göre ölçülür;
- modeldeki her bölen doğrulanır;
- dinlenme açısı `atan2` kullanır;
- N-MPC çarpanları birincil tahminle birlikte kaydırılır (Bölüm 1.3).

Kodun hiçbir yerinde açık matris tersi yoktur. Tek çarpanlara ayırmalar IPOPT (MUMPS) içinde gerçekleşir; IPOPT neredeyse tekil bir KKT matrisini kendisi düzenler ve referans koşusunun 1.200 çözümünün hepsi başarılıdır.

Aşama (aynı 2 çekirdekli makine ve ayarlar)	Önce	Sonra
PID değerlendirme koşusu, telemetriyle 1.200 örnek	5,0 s	0,1–0,2 s
N-MPC değerlendirme koşusu, 1.200 IPOPT çözümü	20,2 s	13,7 s

Aşama (aynı 2 çekirdekli makine ve ayarlar)	Önce	Sonra
`python main.py`, önbellekteki parametrelerle	40,5 s	28,0 s
PID otomatik ayarlama	519 s	142 s
N-MPC otomatik ayarlama	2.039 s	1.181 s

Kazançlar üç değişiklikten gelir:

- derlenmiş CasADi sistemi (örnek başına yaklaşık 70 kat daha hızlı);
- örnek başına sözlükler yerine önceden ayrılmış, vektörel telemetri;
- N-MPC çözücüsünün yeniden kullanımı ve tutarlı biçimde kaydırılmış çarpanlar.

Üç başka fikir ölçüldü ve reddedildi:

- NLP'yi C'ye JIT ile derlemek, yalnızca %25 daha hızlı bir çözüm için ufuk başına 162 s derleme süresi aldı;
- değişkenleri aşama aşama sıralamak kazanç sağlamadı;
- CasADi'nin FATROP çözücüsü bu problemde yakınsamadı.

N-MPC'nin kalan örnek başına süresi IPOPT'un kendi işidir. `tests/test_core.py` içindeki regresyon testleri her düzeltmeyi ve bit düzeyinde aynı sistemi denetler (8 test).

Son eklemeler (14 Ekim 2026, incelemeden sonra)

İncelemeden sonra depoya şu commit'ler eklendi:

Commit	Değişiklik
775af3f	`main.py` için `--controllers` seçeneği: yalnızca PID'yi veya yalnızca N-MPC'yi çalıştırma
761b005	tek kontrolcülü koşu için tek sütunlu `summary.md`
ee02529	N-MPC ufuk taraması çalışması (`studies/horizon_sweep.py`)
8a35bcb	model uyumsuzluğuna dayanıklılık çalışması (`studies/robustness.py`)
0446717	çalışmalar ve tek kontrolcü kipi için belgeler (README §9)

Commit	Değişiklik
f5e5e3c	tüm metin dosyalarının UTF-8 ile okunup yazılması (Windows'taki çökmenin düzeltmesi)
ec9d365, 6dc32ab	dönen rotorlu 3B tekrar sayfası, hedef duruş ve takip hatası vurgusuyla (`studies/animate_3d.py`)

Bölüm 3 — Veri sözlüğü ve metrik tanımları

Her iki kontrolcülü koşu PID klasörüne 16, N-MPC klasörüne 18 ve koşu köküne 12 karşılaştırma dosyası yazar. Hepsi kontrolcü başına örnek başına tutulan tek bir telemetri tablosundan türetilir: 1.200 satır (50 ms ile 60 s), PID için 61, N-MPC için 63 sütun. `--controllers` ile tek bir kontrolcü çalıştırıldığında yalnızca o kontrolcünün klasörü, `settings_used.yaml` ve tek sütunlu bir `summary.md` yazılır; `comparison_*` dosyaları ve `compare_*.png` grafikleri oluşmaz.

Çıktı dosyaları

Dosya	Konum	İçerik
`telemetry.csv`	`PID/` , `N-MPC/`	örnek başına bir satır, kaydedilen her değişken (sözlük aşağıda)
`step_metrics.csv`	`PID/` , `N-MPC/`	referans basamağı başına bir satır: aşım, yükselme/oturma süresi, hatalar, bağlanım
`global_metrics.csv` / `.json`	`PID/` , `N-MPC/`	o kontrolcünün 44 tüm koşu metriği
`controller_parameters.json`	`PID/` , `N-MPC/`	kontrolcünün ayarları ve tam ayar raporu
`nmpc_predictions.npz`	`N-MPC/`	tahmin edilen her yörünge (aşağıda)
`README.txt`	`PID/` , `N-MPC/`	klasörün kısa açıklaması
`00_…png` – `10_…png`	`PID/` , `N-MPC/`	kontrolcü başına grafikler (Bölüm 4)

Dosya	Konum	İçerik
`comparison_timeseries.csv`	kök	iki kontrolcü tek zaman ekseninde: açılar, hatalar, girişler, birikimli IAE ve ∫u², hesaplama süresi
`comparison_step_metrics.csv`	kök	iki basamak tablosu alt alta, `controller` ve `step_id` sütunlarıyla
`comparison_global_metrics.csv`	kök	44 genel metrik, kontrolcü başına bir sütun
`comparison.xlsx`	kök	`global_metrics`, `step_side_by_side`, `step_metrics_long`, `timeseries`, `telemetry_PID`, `telemetry_N-MPC`, `parameters` sayfaları
`compare_01…06.png`	kök	üst üste ve çubuk grafik karşılaştırmaları (Bölüm 4)
`summary.md`	kök	okunabilir sonuç tabloları (tek kontrolcülü koşuda tek sütun)
`settings_used.yaml`	kök	tekrarlanabilirlik için bu koşunun tam ayarları
`trms_3d.html`	kök	**yeni**, isteğe bağlı: `python -m studies.animate_3d` ile üretilen 3B tekrar sayfası (Bölüm 5)

Çalışmalar kendi çıktılarını ayrı bir yere, `outputs/studies/<çalışma>_<damga>/` altına yazar: `horizon_sweep.csv/.md`, `horizon_sweep_tradeoff.png`, `horizon_sweep_tracking.png`, `robustness.csv/.md`, `robustness_iae.png` ve `robustness_tracking.png`. `--publish` ile aynı dosyalar `docs/results/studies/` altına kopyalanır; depodaki yayımlanmış sürümler bunlardır.

`telemetry.csv` sütun sözlüğü

Açılar, adı `_deg` ile bitmedikçe radyan cinsindendir. Her satır bir örneklemenin *başındaki* durumu ve *boyunca* uygulanan girişi tanımlar. 3B sayfa da bu dosyayı okur.

Sütunlar	Anlamı
`time`	örnekleme zamanı t_k [s]
`ref_pitch`, `ref_yaw` (+ `_deg`)	r_k referansları
`x_pitch`, `x_S_v`, `x_u_vv`, `x_yaw`, `x_S_h`, `x_u_hh`	altı durum
`xdot_*`	altı durum türevi f(x, u)
`pitch`, `yaw` (+ `_deg`)	ölçülen açılar (x1 ve x4'ün kopyaları)
`err_pitch`, `err_yaw`	takip hataları e = r − y
`pitch_rate`, `yaw_rate`	gerçek açısal hızlar Omega_v, Omega_h [rad/s]
`pitch_acc`, `yaw_acc`	dS_v/dt, dS_h/dt [rad/s²]
`omega_main_rad_s`, `omega_tail_rad_s` (+ `_rpm`)	rotor hızları
`thrust_main_N`, `thrust_tail_N`	rotor itkileri S_f·F [N]
`J_h`	pitch'e bağlı yaw eylemsizliği [kg·m²]
`torque_aero_pitch`, `torque_gravity_pitch`, `torque_centrifugal_pitch`, `torque_friction_pitch`	dört pitch torku [N·m]
`torque_aero_yaw`, `torque_friction_yaw`	iki yaw torku [N·m]
`u_pitch`, `u_yaw`	doygunluktan sonra uygulanan girişler
`u_unsat_pitch`, `u_unsat_yaw`	kontrolcünün kırpmadan önce istediği giriş (PID: v_k; N-MPC: planlanan ilk giriş)
`du_pitch`, `du_yaw`	önceki örnekten bu yana giriş değişimi
`sat_pitch`, `sat_yaw`	giriş sınırdayken true
`cum_IAE_pitch`, `cum_IAE_yaw`	abs(e)'nin birikimli integrali
`cum_int_u2_pitch`, `cum_int_u2_yaw`	u²'nin birikimli integrali
`compute_time_ms`	kontrolcünün bu örnekte harcadığı duvar saati süresi

Sütunlar	Anlamı
`pid_P_*`, `pid_I_*`, `pid_D_*`	yalnızca PID: üç terim
`pid_integrator_frozen_*`	yalnızca PID: anti-windup etkin
`nmpc_solve_time_ms`, `nmpc_ipopt_iterations`	yalnızca N-MPC: çözücü emeği
`nmpc_success`, `nmpc_return_status`	yalnızca N-MPC: IPOPT yakınsadı mı ve durum metni
`nmpc_cost`	yalnızca N-MPC: bu örneğin probleminin optimal J'si
`nmpc_constraint_violation`	yalnızca N-MPC: en büyük dinamik kısıtı artığı
`nmpc_pred_pitch_next`, `_yaw_next`, `_pitch_end`, `_yaw_end`	yalnızca N-MPC: bir adım sonra ve ufkun sonunda tahmin edilen açılar

`nmpc_predictions.npz`

1.200 örneğin her birinde yapılan planın tamamını tutan sıkıştırılmış bir NumPy arşivi (`numpy.load` ile açılır):

Anahtar	Biçim	Anlamı
`t`	(1200,)	her planın örnekleme zamanı
`X`	(1200, 39, 6)	tahmin edilen durumlar x_0 … x_38
`U`	(1200, 38, 2)	planlanan girişler u_0 … u_37
`R`	(1200, 38, 2)	planın izlediği referanslar

`config/tuned_parameters.json`

Kontrolcü başına bir kayıt. Kaydın hâlâ geçerli olup olmadığına `signature` karar verir (Bölüm 2). Kısaltılmış gerçek bir kayıt:

```
{
  "nmpc": {
    "signature": "6c38…",                    // ayar girdilerinin SHA-256
özeti
    "created_utc": "2026-10-13T11:39:23+00:00",
    "params": {
      "weights": {"q_pitch": 1.0, "q_yaw": 3.049, "q_rate_pitch": 0.00418,
                  "q_rate_yaw": 0.343, "r_pitch": 0.01276, "r_yaw": 0.3523},
      "horizon_steps": 38
    },
    "tuning_report": {
      "method": "scipy.optimize.differential_evolution (parallel, integer
horizon)",
      "objective": 2.3627, "n_function_evals": 224, "generations": 6,
      "wall_time_s": 2039.5, "convergence_history": [2.588, 2.478, …, 2.363],
      "bounds": {"q_yaw_log10": [-1.0, 1.5], …, "horizon_steps": [15, 40]}
    }
  },
  "pid": { "params": {"Kp_pitch": 0.456, "Ki_pitch": 0.396, …}, … }
}
```

(`//` açıklamaları yalnızca anlatım içindir; JSON'un kendisinde açıklama yoktur.) `generations` başlangıç popülasyonundan sonraki nesilleri sayar; N-MPC'nin değerlendirilen 7 popülasyonu için 6 yazmasının nedeni budur. `convergence_history` her nesilden sonraki en iyi J'yi tutar ve `00_tuning_convergence.png` olarak çizilir. Çalışmalar `params` alanını doğrudan bu dosyadan okur.

Metrik tanımları

Gösterim: t_k = k·Ts, Ts = 0,05 s, e_k = r_k − y_k. Her integral dikdörtgen kuralıyla bir toplam olarak hesaplanır: ∫ g dt ≈ Σ g_k·Ts.

Mutlak hatanın integrali (IAE) — `IAE_pitch`, `IAE_yaw`; birim rad·s.

Image — Mutlak hatanın integrali

Yanıt ile referans arasındaki toplam alandır. Küçük ve büyük hataları büyüklükleriyle orantılı sayar ve genel takip kalitesinin en doğrudan tek ölçüsüdür. Havacılık diliyle, araç kendisine söylenen yerde ne kadar uzakta ve ne kadar uzun süre bulunmadı sorusunu ölçer.

Karesel hatanın integrali (ISE) — `ISE_*`; rad²·s.

Image — Karesel hatanın integrali

Kare alma, büyük hataları küçüklerden çok daha fazla cezalandırır; ISE'ye bir basamağın ilk anları gibi en büyük sapmalar hakim olur. Büyük bir sapmanın tehlikeli olduğu durumlarda, örneğin yapısal veya uçuş zarfı sınırlarında, önemlidir.

Zaman ağırlıklı mutlak hatanın integrali (ITAE) — `ITAE_*`; rad·s².

Image — Zaman ağırlıklı mutlak hatanın integrali

Geç hatalar erken hatalardan ağır basar; bu yüzden ITAE yavaş oturmayı ve süren salınımı cezalandırır. Üç çeşidi vardır ve yalnızca t'nin nereden ölçüldüğü farklıdır: tüm koşu metriği koşunun başından itibaren zamanı; basamak başına metrik o basamaktan bu yana geçen zamanı; ayar maliyeti ise herhangi bir eksendeki son değişiklikten bu yana geçen zamanı kullanır.

Karesel ortalama hata (RMSE) — `RMSE_*`; rad.

Image — Karesel ortalama hata

Açının kendi biriminde tipik bir hata büyüklüğüdür ve yorumlanması kolaydır: N-MPC'nin 0,087 rad'lık pitch RMSE'si tipik bir örnekte yaklaşık 5° demektir. ISE gibi büyük hataları daha ağır tartar. `max_abs_error_*` tek en büyük hatayı verir.

Kontrol emeği — `effort_int_u2_*`, `effort_int_abs_u_*`; s.

Image — Kontrol emeği

u gerilimiyle sürülen bir motorda elektrik gücü kabaca u² ile orantılıdır; E_2 çekilen enerjinin bir vekilidir. Bir hava aracında bu batarya kapasitesi veya yakıttır. Burada bu değere, kirişi yerçekimine karşı tutmak için gereken ve iki kontrolcünün de ödemek zorunda olduğu büyük sabit u_v hakimdir.

Girişin toplam varyasyonu (TV) — `total_variation_u_*`; boyutsuz.

Image — Girişin toplam varyasyonu

Girişin yukarı aşağı kat ettiği toplam mesafedir: ortalama düzeyinden bağımsız olarak eyleyicinin ne kadar çalıştırıldığı. Yüksek TV hızlı yön değiştirmeler demektir; bunlar motor ısınmasına, dişli ve rulman aşınmasına, akım sıçramalarına ve hava araçlarında eyleyici bağlantıları ile yapının yorulmasına yol açar. İlgili sütunlar `max_abs_du_*` (tek en büyük sıçrama), `mean_u_*` ve `rms_u_*`'dir.

Doygunluk oranı — `saturation_fraction_*`: iki sınırdan birine eyleyici aralığının %0,05'i kadar yakın olan örneklerin payı: u ⩾ u_max − 0,0005 (u_max − u_min) veya u_min'in aynı mesafe üstü. Burada kullanılan ±1 sınırları için bu u ⩾ 0,999 veya u ⩽ −0,999 demektir. Gerçek sınırlara göre ölçmek, metriği birim olmayan veya asimetrik eyleyiciler için de doğru tutar. Sınırda geçen zaman, beklenmedik bir esintiyi veya bozucuyu bastırmak için hiç yedek yetkinin olmadığı zamandır; sertifikasyon uygulaması bu yüzden genellikle pay ister.

Hesaplama süresi — `compute_time_mean_ms`, `_median_ms`, `_p95_ms`, `_max_ms` ve `realtime_budget_exceeded_fraction`, yani hesaplaması Ts = 50 ms'den uzun süren örneklerin payı. Gerçek bir uçuş bilgisayarında geç kalan bir kontrol güncellemesi kaçırılmış bir son tarihtir. En kötü durum (max, p95) ortalamadan daha önemlidir.

Basamak yanıtı metrikleri

Her j referans basamağı bir eksendeki referansı r_from'dan r_to'ya değiştirir; basamak büyüklüğü Δr = r_to − r_from'dur. Penceresi basamakta başlar ve herhangi bir eksendeki bir sonraki referans değişikliğinde biter. Basamağın işareti ve normalleştirilmiş ilerleme şöyle tanımlanır:

Image — Basamak işareti ve normalleştirilmiş ilerleme

Aşım — `overshoot_pct`. Yanıtın hedefin ne kadar ötesine geçtiği, basamağın yüzdesi olarak:

Image — Aşım

Uçuşta duruş aşımı, aracın komut edilen yönelimin ötesine salınması demektir. Bu bir duruş sınırını ihlal edebilir, bir faydalı yükü sarsabilir veya ikinci bir düzeltme gerektirebilir. `peak_value` ulaşılan uç değerdir. `undershoot_pct` aynı ölçünün yanlış yöndeki karşılığıdır, r_from'a göre: yanıtın önce hedeften ne kadar uzaklaştığı.

Yükselme süresi (%10–%90) — `rise_time_10_90_s`:

Image — Yükselme süresi

Her düzeyin ilk geçildiği an kullanılır. Yanıt hızını ölçer. `delay_time_50_s`, basamaktan yanıtın basamağın yarısını ilk kez kat ettiği ana kadar geçen süredir.

Oturma süresi (%2) — `settling_time_2pct_s`. Yanıtın hedef çevresinde basamak büyüklüğünün ±%2'si genişliğindeki bir bant içinde kalmaya başladığı süre:

Image — Oturma süresi

Pencerenin sonunda yanıt hâlâ bantın dışındaysa basamak "oturmadı" olarak işaretlenir (CSV'de boş veya NaN, `settled = False`). İkiden az sonlu örnek içeren bir pencere, örneğin erken durmuş bir koşudan sonra, tüm basamak metriklerini NaN olarak verir ve oturmadı diye işaretlenir. Bu en pratik tek kalite sayısıdır: bir pilot veya otopilot bir manevraya ancak oturduktan sonra tamamlanmış gözüyle güvenebilir. %2'lik bant sıkıdır; 0,15 rad'lık bir basamak için ±0,003 rad'dır (±0,17°).

Kalıcı durum hatası — `steady_state_error`: pencerenin son 1 s'si boyunca r_to − y'nin ortalaması. Süren bir sapma, aracın yanlış duruşu tuttuğu anlamına gelir.

Basamak başına IAE, ISE, ITAE — basamağın penceresiyle sınırlı integral hatalar; ITAE zamanı basamaktan itibaren ölçülür.

Çapraz bağlanım — `cross_coupling_peak_dev_rad` ve `_deg`. Pencere boyunca *diğer* eksenin kendi (sabit) referansından en büyük sapması:

Image — Çapraz bağlanım tepe sapması

Bir eksendeki manevranın diğer ekseni ne kadar bozduğunu doğrudan ölçer. Pilotlar bu tür etkileşime istenmeyen eksen dışı yanıt der; iş yükünü artırır ve ağır durumlarda uçuş niteliklerini bozar.

Toplanmış basamak metrikleri

`global_metrics` basamak tablosunu eksen başına da özetler: `mean_overshoot_pct_*` ve `max_overshoot_pct_*`; **yalnızca oturan basamaklar** üzerinden hesaplanan `mean_settling_time_s_*` ve `max_settling_time_s_*`; `steps_not_settled_*`; ve `mean_rise_time_s_*`. Ortalama oturma süresini `steps_not_settled` ile birlikte okuyun. PID'nin 6,95 s'lik yaw ortalaması, oturmayan ikisini değil, oturan tek basamağı tanımlar.

Bölüm 4 — Görsel analiz: grafikler nasıl okunur

Tüm karşılaştırma grafikleri aynı kuralları kullanır: mavi PID, turuncu N-MPC, kesikli gri çizgi referanstır; ince dikey gri çizgiler referans değişikliklerini işaretler. Zaman, 60 s'lik değerlendirme boyunca soldan sağa akar. Kod radyanla çalışsa da açılar derece olarak çizilir. Grafiklerdeki etiketler koddan geldiği için İngilizcedir; aşağıdaki açıklamalar onları Türkçe okur.

Her grafikte işe yarayan tek bir ilke vardır. Çizgileri okumadan önce dikey işaretleri bulun ve orada hangi eksenin referansının değiştiğini sorun. *Aynı* eksendeki bir hareket, kontrolcünün komutuna verdiği yanıttır. *Diğer* eksendeki bir hareket ise bir çapraz bağlanım bozucusudur.

Referans takibi (`compare_01_tracking.png`)

Image — Referans takibi, PID ve N-MPC: üstte pitch, altta yaw

Ne gösterir. Üst panel pitch, alt panel yaw'dır. Her panel referansı ve iki kontrolcünün gerçek açılarını üst üste gösterir.

Bir basamak yanıtı nasıl okunur. Panelin kendi eksenine ait her dikey işarette, renkli çizgiyi eski referans düzeyinden yenisine kadar izleyin.

- Ne kadar dik tırmandığı **yükselme süresidir**.
- Kesikli çizginin ne kadar ötesine geçtiği **aşımdır**.
- Kesikli çizginin üzerine yerleşmesine kadar geçen süre **oturma süresidir**.
- Pencerenin sonunda çizgi ile referans arasında kalan boşluk **kalıcı durum hatasıdır**.

t = 0 s'de N-MPC (turuncu) pitch'i dinlenme açısından (−38,5°) 2 s'den kısa sürede −11,5°'ye kaldırır ve orada tutar. PID (mavi) hedefe yaklaşık 3 s'de varır, ardından hedef çevresinde yaklaşık 3 s periyot ve kabaca ±5° genlikle salınır. t = 10 s'deki yaw

basamağında N-MPC yaklaşık 2,5 s'de +28,6°'ye döner; PID aşım yapmadan yavaşça yaklaşarak yaklaşık 10 s'ye ihtiyaç duyar.

Çapraz bağlanım aşımdan nasıl ayırt edilir. Aşım, komut edilen *aynı* eksende, kendi basamağından hemen sonra ve basamak yönünde oluşan bir tümsektir. Çapraz bağlanım ise referansı değişmeyen *diğer* eksendeki bir tümsektir. Pitch paneline t = 10, 30 ve 50 s'de bakın. Bunlar yaw basamaklarıdır; yine de N-MPC'nin pitch'i 17°'ye kadar sıçrar ve yaklaşık 2–3 s içinde toparlanır. Pitch orada yeni bir komut almadığı için bu tümsekler aşım değil, bağlanımdır. Fiziksel nedenleri kuyruk rotorunun tepki momentumudur (Bölüm 1.1'deki 1. bağlanım yolu): N-MPC hızlı dönmek için kuyruk motorunu sert sürer ve kiriş tepki olarak pitch ekseninde hareket eder.

Etki ters yönde de çalışır. t = 0'da ana rotor kirişi kaldırmak için hızlanır ve tepki momentumu kirişi yaw ekseninde döndürür. PID'nin yaw'ı, yaw döngüsü toparlanmadan önce yaklaşık +5°'ye kayar. N-MPC'nin yaw'ı yaklaşık ±2° içinde kalır, çünkü modeli bu tepkiyi öngörür ve büyük bir hata oluşmadan kuyruk rotorunu onu dengeleyecek biçimde çalıştırır.

Kararsızlık nasıl fark edilir. Her salınımın bir öncekinden büyük olduğu büyüyen bir salınım, kararsız bir döngünün imzasıdır. t = 40 s'deki −23° pitch basamağından sonra PID'nin salınımları yaklaşık ±10°'den t = 60 s'ye kadar ±40°'nin ötesine büyür. Bu, Bölüm 1.2'nin öngördüğü gibi, ana rotor itki kazancının en yüksek olduğu çalışma noktasında olur.

Takip hatası (`compare_02_error.png`)

Image — Takip hatası: üstte pitch, altta yaw

Takip grafiğiyle aynı bilgi, referans sıfır çizgisi olacak şekilde yeniden merkezlenmiştir: e(t) = r(t) − y(t). Bu, 60°'lik bir eksende kaybolacak küçük hataları görünür kılar.

- Bir basamakta oluşup sıfıra sönen bir **sıçrama** normal bir geçici durumdur.
- **Basamak uygulanmayan eksendeki** bir sıçrama çapraz bağlanımdır.
- **Sönmeyen bir salınım** bir limit çevrimidir; büyüyorsa kararsızlıktır.
- **Sabit bir kayma** kalıcı durum hatasıdır.

İşarete dikkat edin. Pozitif hata, kirişin referansın *altında* (pitch) veya *gerisinde* (yaw) olduğu anlamına gelir.

Kontrol emeği (`compare_03_control_effort.png`)

Image — Kontrol girişleri ve birikimli emek, PID ve N-MPC

Ne gösterir. Üstteki iki panel ±1 ile sınırlı uygulanan motor girişleridir, u_pitch ve u_yaw. Alt panel birikimli enerji vekilidir, ∫(u_p² + u_y²) dt.

Doygunluk nasıl tanınır. Doygunluk, tam olarak +1 veya −1 boyunca düz giden bir çizgi olarak görünür; sinyal eyleyici sınırında "kırpılmıştır". N-MPC'nin pitch girişi bunu tekrar tekrar yapar: yukarı yönlü her pitch basamağından sonra yaklaşık 1 s boyunca +1'de, t = 30 ve 40 s'de kısa süreler boyunca −1'de kalır. Zamanın %18,8'inde sınırdadır. PID zamanın yalnızca %0,7'sinde sınırdadır: kirişi 0° civarında tutmak neredeyse tam ana rotor girişi gerektirdiğinden t = 37 s ve t = 40 s civarında iki kısa +1 teması. Doygunluk tek başına bir kusur değildir. N-MPC için bilinçlidir, çünkü sınır optimizasyonun bir kısıtıdır ve optimum sıklıkla tam yetkiyi kısa süre kullanmaktır. PID içinse doygunluk, eyleyicinin kontrolcünün istediğini veremediği ve anti-windup'ın devreye girmesi gerektiği anlamına gelir.

N-MPC neden keskin ve hassas darbeler komut eder. Optimizasyon, ana motorun 1,43 s geciktiğini bilir. Kirişi hızlı hareket ettirmek için en iyi plan, motoru aşırı sürmek (giriş +1) ve böylece iç durumunun hızla yükselmesini sağlamak, sonra kirişin hedefe varacağı tahmin edilen anda tam olarak geri çekmektir. Kısa negatif darbe, kirişin hedefi geçip salınmasına izin vermek yerine onu hedefte durduran bir frendir. Kuyruk girişi de aynı "tam itki–karşı itki" deseni gösterir. Bu, zaman-optimal kontrole özgü "bang–bang benzeri" yapıdır: bir yönde tam emek, sonra diğer yönde, sonra tutma.

PID'nin yumuşak dalgaları neden etkisizdir. PID'nin u_pitch'i, pitch hatasıyla aynı 3 s periyotla yaklaşık 0,5 ile 0,65 arasında yumuşakça salınır. Kontrolcü salınan kirişin peşinden koşar: her düzeltme yavaş motor üzerinden, salınım yön değiştirdikten sonra ulaşır; bu yüzden salınımı söndürmek yerine kısmen besler. Dalgalar sakin görünür ama hiç durmaz ve t = 40 s'den sonra giriş aralığının büyük bölümünü kaplayacak kadar büyür.

Birikimli emek eğrisinin okunması. Eğimi anlık u²'dir; dik bir bölüm yoğun eyleme, yatık bir eğim hafif eyleme demektir. Kirişi tutmak her zaman yaklaşık 0,4–0,9'luk bir u_v gerektirdiği için iki eğri de düzenli yükselir. N-MPC'nin çizgisi çoğunlukla her basamaktaki kısa ve dik sıçramalarla daha hızlı tırmanır; bu hızlı manevraların bedelidir. Son değerler N-MPC için 33,7, PID için 25,4'tür.

PID'nin içi (`PID/08_pid_terms.png`, `PID/03_control_inputs.png`)

Image — PID: eksen başına P, I ve D katkıları

Bu grafik PID çıkışını üç terimine ayırır.

- **I (integral)** büyük ve yavaş kısmı taşır: kirişi yerçekimine karşı tutmak için gereken 0,4–0,9'luk trim. Her pitch basamağı onu yeni bir düzeye kaydırır.
- **P ve D** hızlı kısmı taşır ve salınır. D harekete ters yönde çalışır; bu onun sönüm rolüdür. Genliği, motor gecikmesi üzerinden salınımı durdurmaya yetmeyecek kadar küçüktür.
- Uygulanan u, ±1'e kırpılmış toplamdır.

Image — PID: uygulanan ve istenen giriş, birikimli emekle

`03_control_inputs.png` grafiğinde soluk çizgi doygunluktan önce *istenen* giriştir (v_k). Soluk ve koyu çizgilerin ayrıldığı her yerde eyleyici kırpılmış ve anti-windup integratörü sınırlamıştır.

Birikimli IAE (`compare_04_cumulative_iae.png`)

Image — Birikimli mutlak hata: üstte pitch, altta yaw

Ne gösterir. Birikimli integral $\int_0^t \text{abs}(e)\, d\tau$: t anına kadar biriken toplam hata. 60 s'deki son değer tüm koşu IAE'sidir.

Nasıl okunur.

- Herhangi bir andaki **eğim**, o anki abs(e)'ye eşittir.
- **Düz** bir bölüm kusursuz takip, **dik** bir bölüm büyük hata demektir.
- Bir **merdiven** (her basamakta dik yükselme, aralarda düzlük) sağlıklıdır: hatalar yalnızca geçici durumlarda oluşur.
- Hiç düzleşmeyen **düzenli bir eğim**, süren bir hata veya hiç sönmeyen bir salınım demektir.
- **Zamanla dikleşen** bir çizgi hatanın büyüdüğünü, yani kararsızlığı gösterir.

N-MPC'nin pitch eğrisi yaklaşık 1,9 rad·s'de biten temiz bir merdivendir. PID'nin pitch eğrisi sürekli salınım yüzünden hiç düzleşmez ve salınım büyüdükçe t = 40 s'den sonra keskin biçimde dikleşerek yaklaşık 10,2'de biter. Herhangi bir anda eğriler arasındaki dikey fark, o ana kadar bir kontrolcün diğerine karşı biriktirdiği üstünlüktür.

Basamak başına çubuk grafikler (`compare_05_step_metrics.png`)

Image — Basamak başına aşım, oturma süresi, yükselme süresi ve IAE

Dört panel, her referans basamağı için bir çift çubuk içerir (ekseni, zamanı ve başlangıç→hedef açılarıyla etiketlenmiştir). "not settled" (oturmadı) işaretli bir çubuğun oturma süresi yoktur, çünkü yanıt %2'lik bantın içinde hiç kalmamıştır. Bu sonuç, boşluk olarak çizilse de herhangi bir sonlu çubuktan daha iyi değil, daha kötüdür. Aşım ve oturma panellerini birlikte okuyun: düşük aşım ile "not settled" bir arada (birkaç PID yaw basamağında olduğu gibi) 10 s'lik pencere içinde tamamlanamayacak kadar yavaş bir yanıt demektir.

N-MPC açık çevrim tahminleri (`N-MPC/09_nmpc_predictions.png`)

Image — Gerçek yörünge üzerine bindirilmiş N-MPC tahmin ufukları

İnce çizgiler nedir. N-MPC her örneklemede geleceğe doğru 1,9 s'lik (38 örnek) tam bir tahmin edilen yörünge hesaplar. Bu, planladığı girişleri uygularsa beklediği harekettir. Grafik bu tahminlerin her 40'ından birini (toplam 30) yapıldığı andan başlayan ince turuncu bir çizgi olarak çizer. Kalın siyah çizgi gerçekte olan, kesikli çizgi ise referanstır.

Uyuşmanın anlamı. Neredeyse her yerde her ince çizgi tam olarak siyah çizginin üzerindedir. Bu iki şey anlatır. Birincisi model doğrudur: bu simülasyonda sistemle aynı denklemlerdir. İkincisi kontrolcü, hiçbir şey onu şaşırtmadığı için yaptığı planı esasen aynen uygulamıştır.

Ayrışmanın anlamı. İnce bir çizginin siyah çizgiden ayrıldığı yerde planın öngörmediği bir şey olmuştur. En açık örnekler referans basamaklarıdır. t = 9,9 s'de yapılan bir plan sabit kalmayı öngörür; t = 10 s'de yaw referansı sıçrar ve o andan itibaren her plan farklıdır. Görüntüdeki görünür bir örnek t = 1–2 s civarındaki erken tahmindir: hedefe biraz yukarıdan yaklaşmayı bekliyordu, bir sonraki yeniden planlama yolu düzeltti.

“Geleceği görmek” burada gerçekte ne demektir. N-MPC gelecekte *kendi eylemlerinin sonuçlarını* görür. 1,9 s içinde, şimdi itkiyi azaltmanın daha sonra bir düşüş anlamına geldiğini, bir kuyruk rotoru patlamasının kirişi pitch ekseninde hareket ettireceğini ve motorun gecikeceğini bilir. Aşımdan önce fren yapabilmesinin ve bağlanımı büyümeden bastırabilmesinin nedeni bu öngörüdür. Gelecekteki referans değişikliklerini ise *görmez*, çünkü önizleme kapalıdır; her basamağı PID ile aynı anda öğrenir. `reference_preview: true` ile planlar yaklaşan bir basamağa o gelmeden eğilir ve N-MPC manevraya erken başlardı.

N-MPC çözücü istatistikleri (`N-MPC/08_nmpc_solver.png`)

Image — N-MPC çözüm süresi, IPOPT iterasyonları ve optimal maliyet

Üç panel bu öngörünün bedelini gösterir.

- **Örnek başına çözüm süresi.** Noktalı çizgi 50 ms'lik bütçedir. Üstündeki noktalar kaçırılmış son tarihlerdir (referans koşusunda örneklerin %1,0'ı). Duvar saati süreleri bilgisayara bağlıdır ve koşudan koşuya değişir: bu grafikteki en kötü örnek t = 0'da yaklaşık 113 ms iken tekrarlanan koşular yaklaşık 110 ile 310 ms arasında değişti. IPOPT iterasyon sayıları ve tüm yörüngeler koşular arasında aynıdır.
- **IPOPT iterasyonları.** Sıçramalar, önceki plandan gelen sıcak başlatmanın zayıf bir tahmin olduğu ve çözücünün daha çok çalışması gereken referans basamaklarında toplanır.
- **Logaritmik ölçekte optimal maliyet.** Her basamakta sıçrar (maliyete büyük yeni bir hata girer) ve kiriş oturdukça kat kat azalır.

Bu sıçramaları takip grafiğiyle karşılaştırmak, hesaplama baskısının tam da kontrolün en çok önem taşıdığı anlara denk geldiğini gösterir.

Bölüm 5'teki çalışma grafikleri (`horizon_sweep_*.png` , `robustness_*.png`) ve 3B sayfa da aynı renk kurallarını izler; okunuşları orada açıklanır.

Bölüm 5 — N-MPC çalışmaları ve 3B web simülasyonu

Bu bölüm, İngilizce rehberden sonra depoya eklenen üç aracı anlatır. İkisi N-MPC'yi daha derinden inceleyen bağımsız çalışmalardır: ufuk taraması ve model uyumsuzluğuna dayanıklılık. Üçüncüsü, bir simülasyon koşusunu tarayıcıda 3B olarak oynatan web

sayfasıdır. Üçü de `studies/` paketindedir, ana kodu değiştirmez ve önbellekteki ayarlanmış parametreleri yalnızca okur.

Çalışmaların ortak kurulumu

Her iki çalışma da ana koşuyla aynı 60 s'lik değerlendirme senaryosunu, aynı Ts = 0,05 s'yi ve önbellekteki N-MPC ağırlıklarını kullanır; her seferinde yalnızca tek bir şeyi değiştirir. Koşular `studies/common.py` içindeki `run_many` ile paralel süreçlerde çalışır. Bir koşu şu durumlarda ıraksamış sayılır: durum sonlu olmaktan çıktığı için erken durduysa, |pitch| 1,4 rad'ı aştıysa veya |yaw hatası| 1,5 rad'ı aştıysa. Sonuçlar `outputs/studies/<çalışma>_<damga>/` altına bir CSV, okunabilir bir Markdown tablosu ve iki grafik olarak yazılır. `--publish` aynı dosyaları `docs/results/studies/` altına kopyalar; aşağıdaki tablolar oradaki yayımlanmış sonuçlardır. `--workers` paralel süreç sayısını belirler (varsayılan: tüm çekirdekler).

Ufuk taraması (`python -m studies.horizon_sweep`)

Soru. Kontrolcü geleceğe ne kadar uzak bakmalıdır? Ayarlanan ağırlıklar sabit tutulur ve yalnızca tahmin ufku N değiştirilir. Varsayılan değerler N = 10, 15, 20, 25, 30, 38 ve 50'dir; `--horizons` ile başka değerler verilebilir.

N	Ufuk	Sonuç	IAE pitch / yaw [rad·s]	En büyük pitch aşımı [%]	Ortalama / en uzun çözüm [ms]	Bütçe aşımı
10	0,50 s	ıraksadı	20,1 / 38.500	116	3,9 / 47	%0,0
15	0,75 s	ıraksadı	14,3 / 13.800	56,7	4,6 / 57	%0,1
20	1,00 s	ıraksadı	12,4 / 2.790	0,3	6,6 / 50	%0,0
25	1,25 s	kararlı, tüm basamaklar oturur	3,23 / 2,00	4,3	6,9 / 83	%0,3
30	1,50 s	kararlı	2,09 / 1,92	3,8	7,3 / 127	%0,8
38 (ayarlanan)	**1,90 s**	**kararlı**	**1,88 / 1,87**	**1,0**	**7,7 / 80**	**%0,6**
50	2,50 s	kararlı	1,82 / 1,85	1,0	9,2 / 166	%0,7

Image — Ufuk taraması: takip hatası ve çözüm süresi arasındaki ödünleşim

Grafik nasıl okunur. Yatay eksen örnek sayısı olarak ufuk N'dir. Sol eksen pitch (turuncu daireler) ve yaw (mor kareler) IAE'sini, sağ eksen IPOPT çözüm süresini verir: ortalama (yeşil kesikli) ve %95'lik dilim (yeşil noktalı). Kırmızı gölgeli şeritler ıraksayan ufuklardır ("diverged"); IAE'leri binlerle ifade edildiği için çizilmez. Üstteki kesikli kırmızı çizgi 50 ms'lik bütçe, N = 38'deki noktalı dikey çizgi ayarlanan ufuktur. İyi bir ufuk, IAE eğrisinin düzleştiği ama çözüm süresinin henüz fazla artmadığı noktadadır.

Image — Ufuk taraması: seçilmiş ufuklar için pitch ve yaw takibi

Ne öğretir. Üç sonuç öne çıkar.

1. **Bir alt sınır vardır.** Kontrolcünün kirişi tutabilmesi için yaklaşık 1,25 s ileriyi görmesi gerekir. Bu, ana motorun 1,43 s'lik zaman sabitine yakındır: plan, bir komutun motor üzerinden kirişe ulaşıp sonuç vermesini görecek kadar uzun olmalıdır. Daha kısa ufuklarla kontrolcü her adımda "şimdilik iyi görünen" ama birkaç saniye sonra felakete yol açan eylemler seçer ve döngü ıraksar.
2. **Getiri hızla azalır.** N = 25'ten 38'e geçmek toplam IAE'yi 5,23'ten 3,75'e indirir. N = 38'den 50'ye geçmek ise yalnızca yaklaşık %3 kazandırır, buna karşılık ortalama çözüm süresi yaklaşık %20 artar.
3. **Ayar doğrulanmış olur.** Diferansiyel evrimin seçtiği N = 38, bu ödünleşimin dirseğindedir. Ayarlayıcı 15–40 aralığında aradığı için daha büyük ufukları denememişti; tarama bunu tamamlar.

Duvar saati çözüm süreleri koşudan koşuya ve makineden makineye değişir; yörüngeler ve metrikler ise deterministiktir.

Model uyumsuzluğuna dayanıklılık (`python -m studies.robustness`)

Soru. Ana karşılaştırmada N-MPC'nin tahmin modeli ile simüle edilen sistem aynı denklemleri ve parametreleri paylaşır; bu N-MPC'yi kayırır. Gerçek bir TRMS'nin parametreleri makaledekilerden farklı olduğunda ne olur? Bu çalışmada kontrolcü **nominal** modeli ve ayarlanmış N, Q, R değerlerini korur; yalnızca simüle edilen "gerçek" sistemin parametreleri bozulur. Her durum tek bir fiziksel etkiyi değiştirir, ayrıca birleşik bir en kötü durum da vardır. Bozulmuş parametreler `dataclasses.replace` ile türetildiği için `TRMSParams` doğrulaması onlara da uygulanır.

Durum	Değişiklik	IAE pitch / yaw [rad·s]	Toplam IAE / nominal	En büyük pitch aşımı [%]	Oturmayan basamak (pitch / yaw)	u_pitch doygun
nominal	sistem = model	1,88 / 1,87	1,00	1,0	0 / 0	%18,8
kiriş eylemsizliği +%20	J_v × 1,2	1,89 / 1,89	1,01	6,9	0 / 0	%18,3
kiriş eylemsizliği −%20	J_v × 0,8	2,02 / 1,88	1,04	3,5	0 / 0	%18,4
rotor itkisi −%15	S_f × 0,85 (iki rotor da zayıf)	4,63 / 2,78	1,98	11,2	3 / 3	%31,6
rotor itkisi +%15	S_f × 1,15 (iki rotor da güçlü)	4,24 / 2,64	1,84	80,9	3 / 2	%12,2
sürtünme × 2	K_v, K_h × 2	1,81 / 1,89	0,99	0,2	0 / 0	%18,2
ana motor %30 daha yavaş	T_mr × 1,3	2,25 / 1,94	1,12	6,6	0 / 0	%28,2
karşı ağırlık +%5	A × 1,05 (dinlenme açısı kayar)	10,9 / 4,58	4,12	208	3 / 3	%13,5
birleşik	J_v × 1,2; S_f × 0,85; T_mr × 1,3; K_v, K_h × 2	4,75 / 2,82	2,02	18	3 / 3	%34,5
makaledeki kuyruk katsayısı	F_h doğrusal terimi 0,0801 → 0,8080	4,15 / 4,34	2,27	47,8	3 / 3	%52,6

Hiçbir durumda döngü ıraksamaz.

Image — Dayanıklılık: durum başına takip hatası

Grafik nasıl okunur. Her satır bir sistem durumudur; turuncu çubuk pitch IAE'sini, mor çubuk yaw IAE'sini verir. Noktalı dikey çizgi nominal düzeydir. Bu çizgiye yakın biten çubuklar, kontrolcünün o model hatasını neredeyse hiç bedel ödemeden tolere ettiğini gösterir.

Image — Dayanıklılık: nominal ve zor durumlar için pitch ve yaw takibi

İkinci grafik nominal durumu (kalın turuncu) kararlı kalan en zor üç durumla üst üste çizer. Karşı ağırlık +%5 durumunda (mor) pitch her basamakta referansın yaklaşık 6–14° üstünde, yaw da birkaç derece yanında oturur: kalıcı sapmanın tipik görünümü. Makaledeki kuyruk katsayısında (yeşil) iki eksen de sönmeyen sabit genlikli bir salınım, yani bir limit çevrimi gösterir. Birleşik durum (pembe) nominalin biraz altında kalır ve basamaklarda daha büyük sarsıntılar yaşar.

Ne öğretir. Sonuçlar üç gruba ayrılır.

- **Dinamik uyumsuzluk zararsızdır.** Kiriş eylemsizliğinde ±%20, iki kat sürtünme ve %30 daha yavaş bir ana motor toplam IAE'yi en fazla %12 artırır ve her basamak hâlâ oturur. Bunlar sistemin ne kadar *hızlı* tepki verdiğini değiştirir; her örneklemede yeniden planlama bu farkı kolayca kapatır.
- **Statik kazanç uyumsuzluğu kalıcı bir sapma bırakır.** Rotor itkisinde ±%15 veya %5 daha ağır bir karşı ağırlık, belirli bir açıyı tutmak için gereken girişi değiştirir. Kontrolcünün modeli yanlış trim girişini tahmin eder. N-MPC'nin integral etkisi veya bozucu kestirimi olmadığından, referansın üzerine değil yanına oturur. %2'lik bant içinde hiçbir basamak oturmaz ve toplam IAE 1,8 (itki) ile 4,1 kat (karşı ağırlık) artar. Karşı ağırlık en hassas parametredir, çünkü dinlenme açısını ve tüm yerçekimi torkunu doğrudan değiştirir.
- **Makaledeki kuyruk katsayısı.** Basılmış 0,8080 değeriyle kuyruk rotoru modelin beklediğinden çok daha güçlü itki üretir ve döngü sürekli bir salınıma girer; pitch girişi zamanın %52,6'sında doygundur. Bu, kodda kullanılan 0,0801 düzeltmesini destekler (Bölüm 1.1).

Sonraki doğal adım sapmasız (offset-free) MPC'dir: ölçümlerden kestirilen ek bir bozucu durumu modele eklenir ve kontrolcü bu sabit hatayı kendisi telafi eder. Bu, PID'nin integral teriminin MPC'deki karşılığıdır.

Bu çalışma ana karşılaştırmanın sonucunu tersine çevirmez: en kötü durumda bile N-MPC'nin toplam IAE'si (nominalin 4,12 katı, yaklaşık 15,5 rad·s) nominal PID'ninkinden (18,50) düşüktür. Ancak bu karşılaştırma eşit değildir; PID de aynı bozulmuş sistemde test edilmemiştir.

3B web simülasyonu (`python -m studies.animate_3d --open`)

`studies/animate_3d.py`, `python main.py` ile üretilen bir koşunun `telemetry.csv` dosyalarını okur ve koşuyu tarayıcıda 3B olarak oynatan tek, kendi kendine yeten bir HTML sayfası üretir: `<koşu>/trms_3d.html`. three.js kütüphanesi sayfanın içine gömüldüğü için

sayfa internet bağlantısı olmadan da çalışır; dosya e-postayla gönderilebilir veya herhangi bir tarayıcıda açılabilir. Sayfanın arayüzü Türkçedir.

Image — 3B web simülasyonu, N-MPC koşusu, t ≈ 33,7 s (yaw basamağından hemen sonra)

Komutlar.

Komut	Ne yapar
`python -m studies.animate_3d`	`outputs/` içindeki en son koşudan sayfayı üretir
`python -m studies.animate_3d --run outputs/run_X`	belirli bir koşudan üretir
`python -m studies.animate_3d --open`	sayfayı üretir ve varsayılan tarayıcıda açar
`--out <dosya>`	çıktı dosyasının yolunu değiştirir
`--fragment`	sayfayı başka bir sayfaya gömülebilecek bir parça olarak üretir

Koşu iki kontrolcüyü de içeriyorsa sayfada bir N-MPC / PID anahtarı bulunur; `--controllers` ile tek kontrolcülü bir koşudan üretilen sayfada yalnızca o kontrolcü vardır.

Sahne. Kiriş, telemetrideki pitch ve yaw açılarıyla birebir hareket eder. Turuncu ana rotor pitch'i, mavi kuyruk rotoru yaw'ı sürer. Yarı saydam gri bir "hayalet" kiriş, **HEDEF** etiketiyle referans duruşu gösterir; gerçek kirişle arasındaki açı "hata X°" etiketiyle yazılır. İtki okları her rotorun itkisinin yönünü ve büyüklüğünü gösterir. Fareyle sürükleme sahneyi döndürür, tekerlek yakınlaştırır, boşluk tuşu oynatır veya durdurur.

Yan kartlar.

- **Takip hatası (hedef − gerçek):** pitch ve yaw hatası derece olarak. İki hata da ±2° içindeyse "Hedefte ✓ (±2° içinde)", değilse "Hedefte değil" yazar.
- **Açılar:** pitch ve yaw'ın gerçek ve hedef değerleri.
- **Rotorlar:** ana ve kuyruk rotorunun gerçek hızı (rpm) ve itkisi (N). Kuyruk rotoru yön değiştirdiğinde hızı ve itkisi negatif olur.
- **Kontrol girişleri:** u_v (ana motor) ve u_h (kuyruk motoru) çubukları; giriş ±1 sınırındayken bir **SINIRDA (doygun)** işareti çıkar.

Kontrol çubuğu. Oynat/Durdur düğmesi; zaman kaydırıcısı; Hız seçimi (0,25×–4×); Pervane görünümü (gerçek hızın 1/100'ü, 1/50'si veya 1/20'si); N-MPC/PID anahtarı; Hedef kiriş ve İtki okları onay kutuları; Görünümü sıfırla düğmesi.

Zaman grafiği. Sayfanın altındaki şerit grafik pitch'i (turuncu) ve yaw'ı (mavi) hedefle (beyaz kesikli) birlikte çizer. Gerçek açı ile hedef arasındaki gölgeli alan takip hatasıdır, yani Bölüm 3'teki IAE'nin görsel karşılığıdır. Dikey çizgi oynatma anını gösterir; grafiğe tıklamak o ana atlar.

Pervaneler neden yavaşlatılır. Gerçek rotor hızları yaklaşık 5.700 rpm (ana) ve 14.000 rpm'ye (kuyruk) ulaşır. Bu, 60 Hz'lik bir ekranın gösterebileceğinden çok hızlıdır: kanatlar kare başına birçok tur döner ve göz onları duruyor ya da geriye dönüyor gibi görür ("vagon tekerleği" etkisi). Bu yüzden kanatlar gerçek hızın seçilen bir kesriyle çizilir; gerçek rpm yan kartta yazar ve hızla büyüyen yarı saydam bir disk dönüş hızını ayrıca gösterir.

Sayfada neye bakmalı. Ekran görüntüsü N-MPC'yi t ≈ 33,7 s'de, t = 30 s'deki −28,6°'lik yaw basamağından hemen sonra gösterir. Yaw hedefine neredeyse ulaşmıştır (−1,1° hata), ama pitch, referansı hiç değişmediği hâlde 3° hatalıdır. Bu, Bölüm 1.1'deki 1. bağlanım yolunun canlı görüntüsüdür: kuyruk rotoru yaw dönüşü için ters yönde (−3.944 rpm) dönerken tepki momentumu kirişi pitch ekseninde sarsar. Aynı anı PID'ye geçerek izlemek, Bölüm 4'teki salınımı ve yavaş yaw yaklaşımını üç boyutta gösterir.

Bölüm 6 — Sonuçlar, sınırlamalar ve sözlük

Simülasyonda N-MPC, PID'den yaklaşık 5 kat daha doğru takip yaptı (toplam IAE 3,75'e karşı 18,50 rad·s). Bunun bedeli %32 daha fazla kontrol emeği, 4 kat daha fazla giriş değişimi ve örnek başına yaklaşık 200 kat daha fazla hesaplamaydı. Yeni çalışmalar bu sonucu iki yönden sınırlar: ufuk en az yaklaşık 1,25 s olmalıdır ve statik kazanç hataları N-MPC'nin üstünlüğünü belirgin biçimde azaltır.

Tüm koşu sonuçları

`docs/results/comparison_global_metrics.csv` dosyasındaki değerler (değerlendirme senaryosu, 60 s):

Metrik	PID	N-MPC	Daha iyi
IAE pitch / yaw [rad·s]	10,21 / 8,29	1,88 / 1,87	N-MPC
ISE pitch / yaw [rad²·s]	3,68 / 3,63	0,45 / 0,92	N-MPC

Metrik	PID	N-MPC	Daha iyi
ITAE pitch / yaw [rad·s²]	386 / 263	43,8 / 56,1	N-MPC
RMSE pitch / yaw [rad]	0,248 / 0,246	0,087 / 0,124	N-MPC
En büyük mutlak hata pitch / yaw [rad]	1,04 / 1,01	0,53 / 1,00	N-MPC (pitch)
Oturan basamaklar (pitch / yaw)	3′te 0 / 3′te 1	3′te 3 / 3′te 3	N-MPC
Ortalama aşım pitch / yaw [%]	44,5 / 0,2	0,6 / 2,6	pitch′te N-MPC, yaw′da PID
Ortalama yükselme süresi pitch / yaw [s]	2,55 / 5,83	0,92 / 0,98	N-MPC
Toplam emek ∫u²	25,4	33,7	PID
Toplam giriş varyasyonu	11,0	44,6	PID
Tek en büyük pitch giriş sıçraması [–]	0,16	1,92	PID
Doygun örnekler, pitch	%0,67	%18,8	PID
Hesaplama süresi ortalama / p95 [ms]	0,06 / 0,09	10,8 / 20,1	PID
50 ms bütçeyi aşan örnekler	%0	%1,0	PID

En büyük yaw hatası iki kontrolcü için de yaklaşık 1,0 rad′dır. Bu yalnızca en büyük yaw basamağının boyudur (t = 30 s′de +0,5′ten −0,5 rad′a); referansın sıçradığı anda, hiçbir kontrolcü tepki veremeden oluşur.

Basamak başına sonuçlar

Basamak	PID aşım [%]	PID oturma [s]	N-MPC aşım [%]	N-MPC oturma [s]	N-MPC'nin diğer eksene bağlanımı [°]
Pitch −38,5° → −11,5°, 0 s′de	25,5	oturmadı	0,2	1,75	1,8
Yaw 0° → 28,6°, 10 s′de	0,0	oturmadı	2,4	2,50	17,0

Basamak	PID aşım [%]	PID oturma [s]	N-MPC aşım [%]	N-MPC oturma [s]	N-MPC'nin diğer eksene bağlanımı [°]
Pitch −11,5° → −2,9°, 20 s'de	31,9	oturmadı	0,6	1,80	1,1
Yaw 28,6° → −28,6°, 30 s'de	0,7	6,95	5,5	4,10	14,9
Pitch −2,9° → −22,9°, 40 s'de	76,0	oturmadı	1,0	1,30	2,5
Yaw −28,6° → 0°, 50 s'de	0,0	oturmadı	0,0	1,85	14,9

İki desen öne çıkar.

- **Asimetrik bağlanım.** N-MPC'nin pitch basamakları yaw'ı neredeyse hiç bozmaz (1–2,5°). Yaw basamakları ise pitch'i yaklaşık 15–17° bozar, çünkü kuyruk rotorunun tepki momentumu doğrudan pitch hızına etki eder. Bu, sistemin ve ayarlanmış ağırlıkların ($q_y \approx 3 \times q_p$) bir özelliğidir, çözücünün bir kusuru değildir. 3B sayfada bu sarsıntı yaw basamaklarından sonra kolayca izlenebilir.
- **PID aşımı limit çevrimiyle şişer.** PID'nin pitch aşımı, basamak geldiğinde zaten salınmakta olan bir kiriş üzerinde ölçülür. Aynı nedenle t = 20 s'deki `undershoot_pct` değeri %69'dur: kiriş o anda bir salınımın dibindeydi.

Yeni çalışmaların özeti

Çalışma	Ana bulgu
Ufuk taraması	$N \leqslant 20$ ($\leqslant 1{,}0$ s) ıraksar; N = 25 (1,25 s) kararlıdır; N = 38'in ötesinde kazanç yaklaşık %3, çözüm süresi artışı yaklaşık %20'dir
Dayanıklılık	dinamik hatalar (eylemsizlik, sürtünme, motor hızı) en fazla +%12 IAE; statik kazanç hataları (itki, karşı ağırlık) kalıcı sapma ve 1,8–4,1 kat IAE; makaledeki kuyruk katsayısı limit çevrimi
3B web simülasyonu	sonuçları ve bağlanımı görsel olarak doğrulamak için çevrimdışı çalışan, Türkçe arayüzlü tekrar sayfası

Karşılaştırma neyi gösterir, neyi göstermez

Mükemmel bir model ve tam durum ölçümüyle, iyi ayarlanmış bir N-MPC'nin bu sistemin düşük sönümünü, motor gecikmesini, değişken kazancını ve bağlanımını, iyi ayarlanmış, sabit kazançlı, merkezi olmayan bir PID'den çok daha iyi yönettiğini gösterir. Dayanıklılık çalışması, bu üstünlüğün makul dinamik model hatalarında korunduğunu da gösterir.

N-MPC'nin donanımda aynı farkla kazanacağını **göstermez**. Başlıca sınırlamalar:

1. **Statik kazanç uyumsuzluğu.** Dayanıklılık çalışması model hatasının bedelini ölçtü: itki veya karşı ağırlık gibi statik parametrelerdeki küçük hatalar kalıcı sapma bırakır. Sapmasız MPC (bozucu kestirimi) eklenmeden donanımda aynı hassasiyet beklenmemelidir. PID ise integral terimi sayesinde bu tür sapmaları kendiliğinden giderir ve aynı bozulmuş sistemlerde henüz test edilmemiştir.
2. **Tam durum, gürültü yok.** N-MPC altı durumun tümünü tam olarak okur. Gerçek bir TRMS yalnızca iki açıyı, nicemlemeli kodlayıcılarla ölçer; bir gözlemci gerekir.
3. **Saf PID.** TRMS pitch'i için standart çözümler olan yerçekimi ileri beslemesi ve kazanç programlama bilinçli olarak dışarıda bırakıldı.
4. **Doğrulanmamış düzeltme.** Kuyruk itki katsayısı (makaledeki 0,8080'e karşı 0,0801) laboratuvar kılavuzuyla karşılaştırılmalıdır. Dayanıklılık çalışması düzeltmeyi destekler, ancak kanıtlamaz.
5. **Gerçek zaman payı.** IPOPT örneklerin yaklaşık %1–1,5'inde 50 ms'lik son tarihi kaçırır. Ufuk taraması bunun ufukla birlikte büyüdüğünü gösterir.
6. **Sınırlı ayar bütçesi.** İki arama da iterasyon sınırında durdu; DE global optimumu garanti etmez.
7. **Tek senaryo.** Sonuçlar tek bir 60 s'lik basamak dizisi içindir. Sinüzoidal takip ve dış bozucular dayanıklılığı daha da sınar.

Sonraki adımlar

Depodaki çalışmaların doğal devamı şunlardır: sapmasız MPC (bozucu durumu kestirimi); yalnızca açıları ölçen bir genişletilmiş Kalman filtresi; PID'nin de dayanıklılık senaryolarında test edilmesi; yerçekimi ileri beslemeli bir PID'nin üçüncü kontrolcü olarak eklenmesi (Bölüm 2'deki arayüz bunu tek bir sınıfa indirger); ve gerçek zamanlı iterasyon (RTI) ile sınırlı çözüm süresi.

Sözlük

Terim	Anlamı
Anti-windup	eyleyici doygunken PID integratörünün büyümesini durduran mantık
Açgözlü seçim (greedy selection)	DE'de aday ile mevcut üyeden daha iyi olanın tutulması
Bağlanım (cross-coupling)	bir girişin veya eksenin başka bir eksene etkisi
CasADi	sembolik modelleme, otomatik türev alma ve optimizasyon için açık kaynaklı kütüphane
Çoklu atış (multiple shooting)	optimal kontrol problemini durumlar ve girişler değişken olacak, dinamik kısıtlarıyla bağlanacak şekilde kurmak
Denge / trim	hiçbir şeyin değişmediği bir durum ve bunu sağlayan sabit giriş
Diferansiyel evrim (DE)	popülasyon tabanlı, türev gerektirmeyen global optimizasyon yöntemi
Doygunluk (saturation)	eyleyicinin fiziksel sınırında tutulması
İmza (önbellek)	önbellekteki bir sonucun hesaplandığı girdileri tanımlayan özet (hash)
IPOPT	büyük doğrusal olmayan programlar için iç nokta çözücüsü
Kayan ufuk (receding horizon)	ileri kayan bir pencere üzerinde her örneklemede yeniden planlama
KKT koşulları	kısıtlı bir optimizasyonun birinci dereceden optimallik koşulları
Limit çevrimi	doğrusal olmayan bir döngüde sabit genlikli sürekli salınım
Merkezi olmayan kontrol	etkileşimleri yok sayan, eksen başına bağımsız kontrolcüler
MIMO / SISO	çok girişli çok çıkışlı / tek girişli tek çıkışlı

Terim	Anlamı
Model uyumsuzluğu	kontrolcünün modeli ile gerçek sistem arasındaki fark
NLP	doğrusal olmayan program: maliyeti veya kısıtları doğrusal olmayan optimizasyon
Otomatik türev alma	türevlerin, program işlemlerine zincir kuralı uygulanarak bilgisayarda tam hesaplanması
Oturma süresi	yanıtın hedef çevresindeki bir bantta (burada ± %2) kalmaya başladığı süre
RK4	dördüncü dereceden Runge–Kutta sayısal integrasyonu
Sıcak başlatma (warm start)	bir optimizasyonu önceki çözümden başlatmak
Sıfırıncı derece tutucu (zero-order hold)	sayısal bir girişi örnekler arasında sabit tutmak
Sönüm oranı (zeta)	salınımların ne kadar hızlı söndüğü; 1 = aşım yok, 0'a yakın = uzun salınım
Sapmasız MPC (offset-free MPC)	sabit model hatalarını bir bozucu durumu kestirerek telafi eden MPC
Tahmin ufku	MPC'nin üzerinde plan yaptığı gelecek örnek sayısı (N = 38, 1,9 s)
three.js	tarayıcıda WebGL ile 3B grafik çizen JavaScript kütüphanesi

Kaynaklar

- arXiv:1306.4883, "Fault-Tolerant Control of a 2 DOF Helicopter (TRMS System) Based on H∞" — model yapısı ve parametreler.
- Proje deposu: github.com/aybarsavci/TRMS-Kontrol — kod, önbellek, referans sonuçlar ve `docs/results/studies/` altındaki çalışma sonuçları (commit 6dc32ab).
- three.js — 3B web simülasyonunda kullanılan kütüphane (MIT lisansı, `studies/vendor/THREE_LICENSE.txt`).